

AI Model Hub Compression

DS5110: Big Data Systems

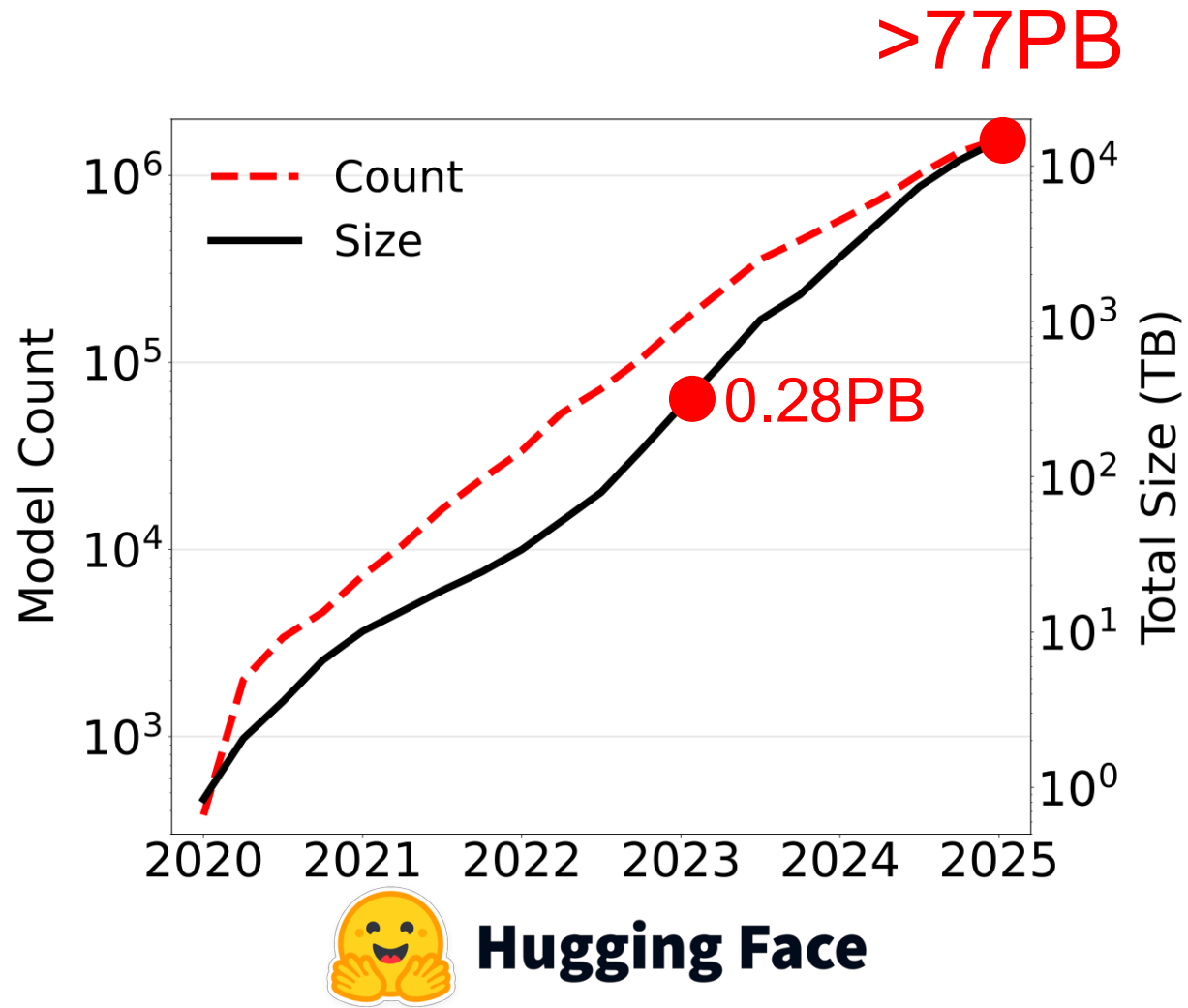
Summer 2026

Lecture 11

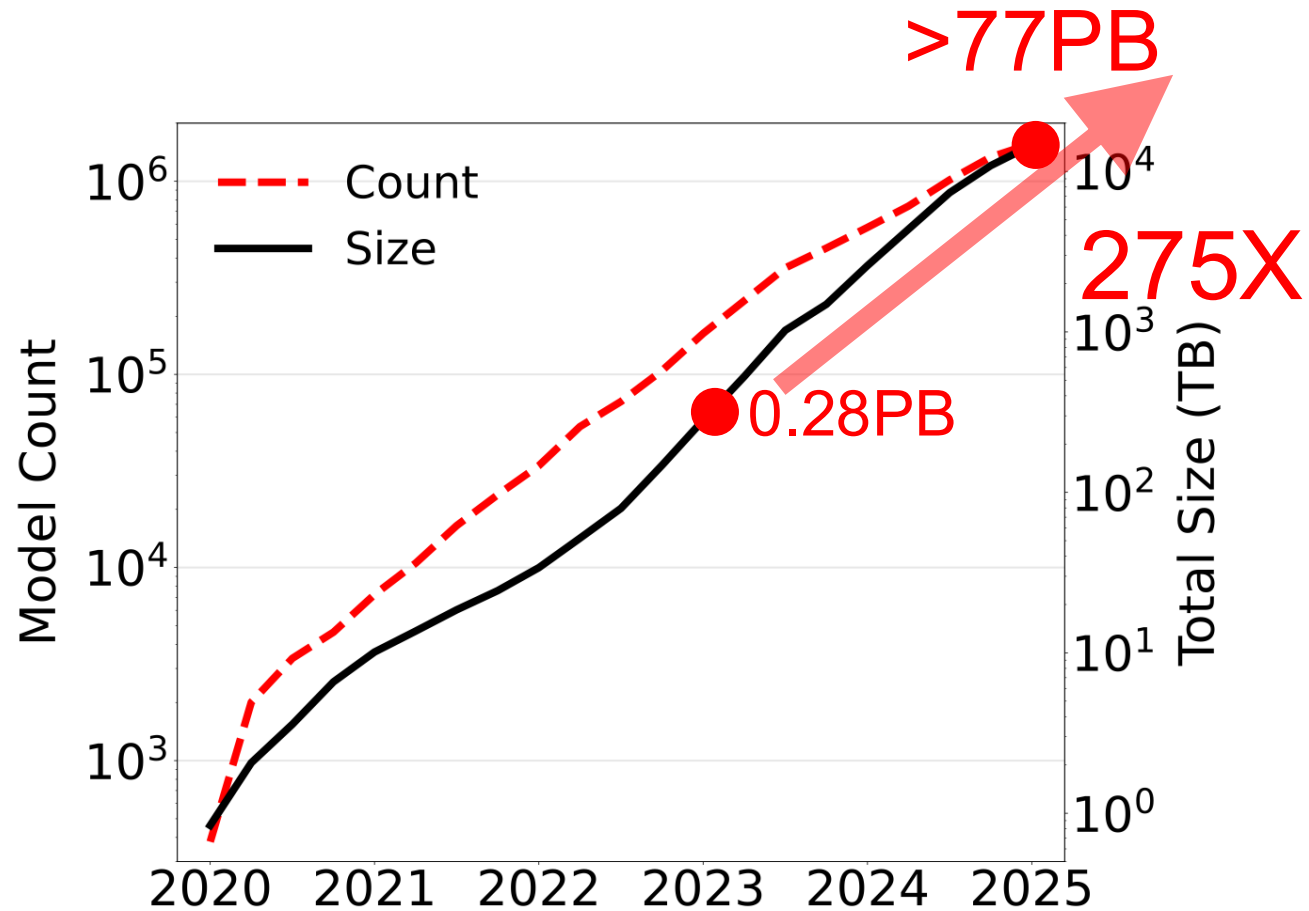
Yue Cheng



Model hub storage is growing unsustainably



Model hub storage is growing unsustainably

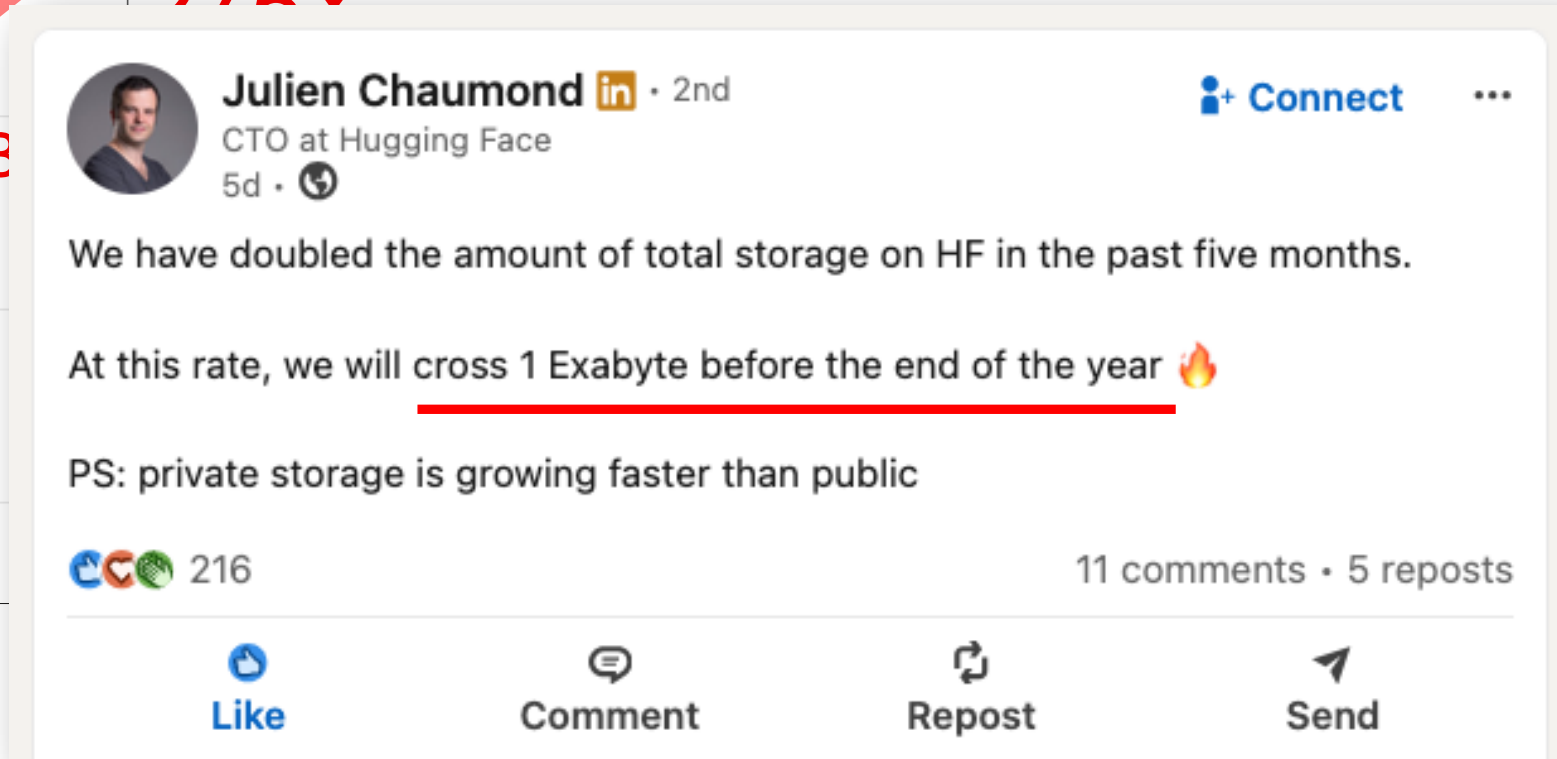
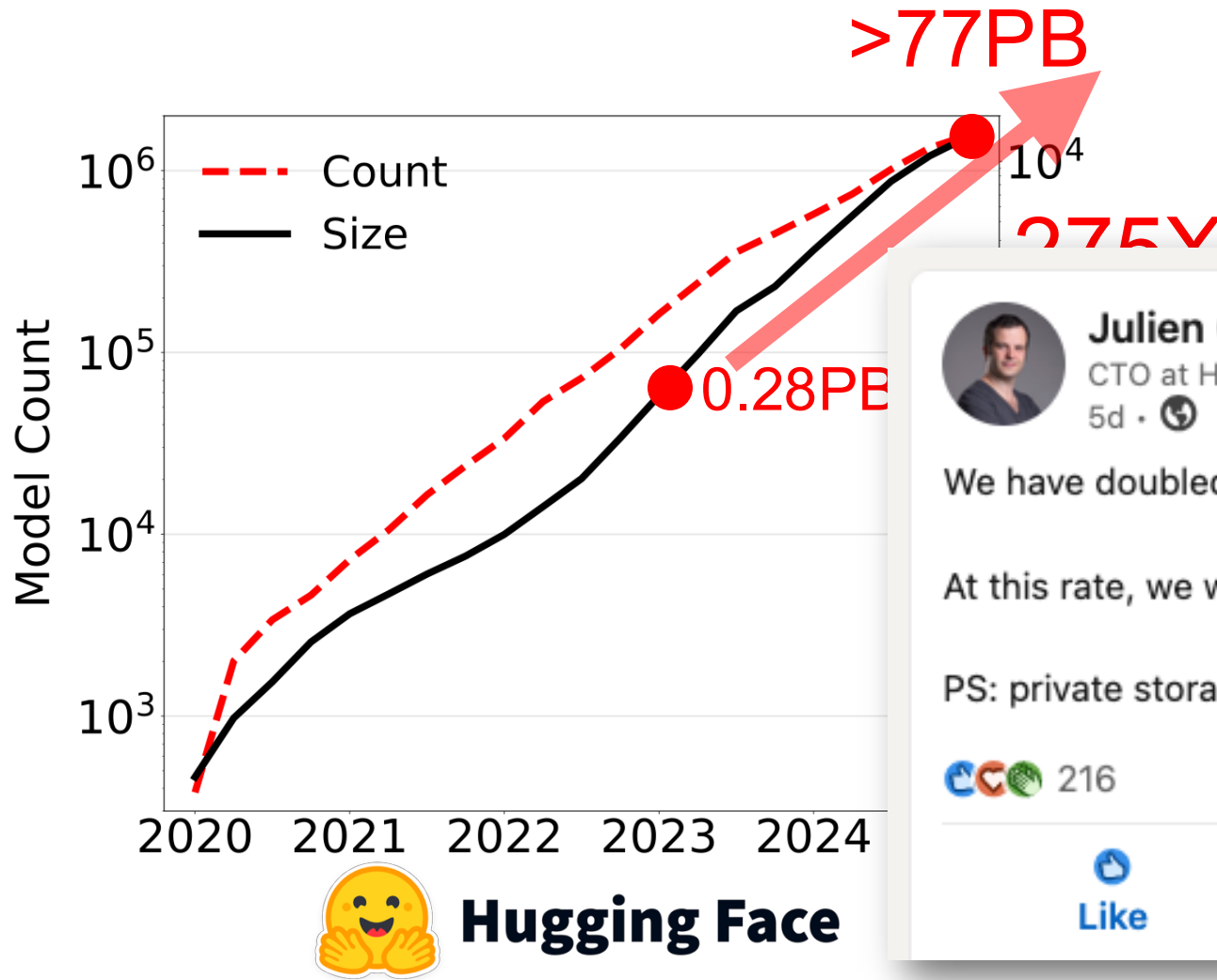


Hugging Face

Exponential growth

1EB model storage by 2027

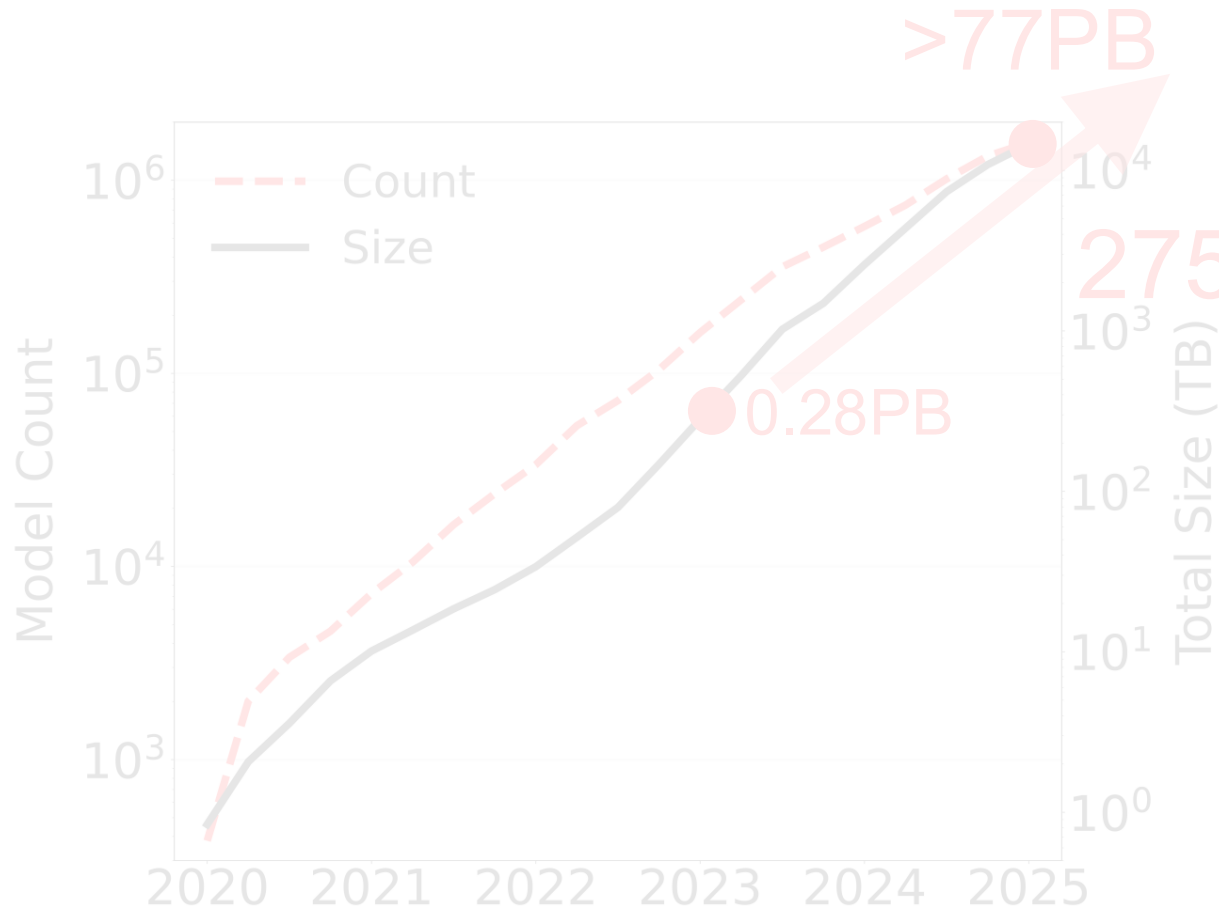
Model hub storage is growing unsustainably



Exponential growth

Y. Cheng 1EB model storage by 2027 → end of year 2026

Checkpoint storage is growing unsustainably



Hugging Face

Exponential growth

1EB model storage by 2027

Y. Cheng

100B model → **1TB** checkpoint

1T model → **15TB** checkpoint

∞ Meta

16K GPUs → **187-min MTTF**

used a **240 PB** storage fabric for pretraining

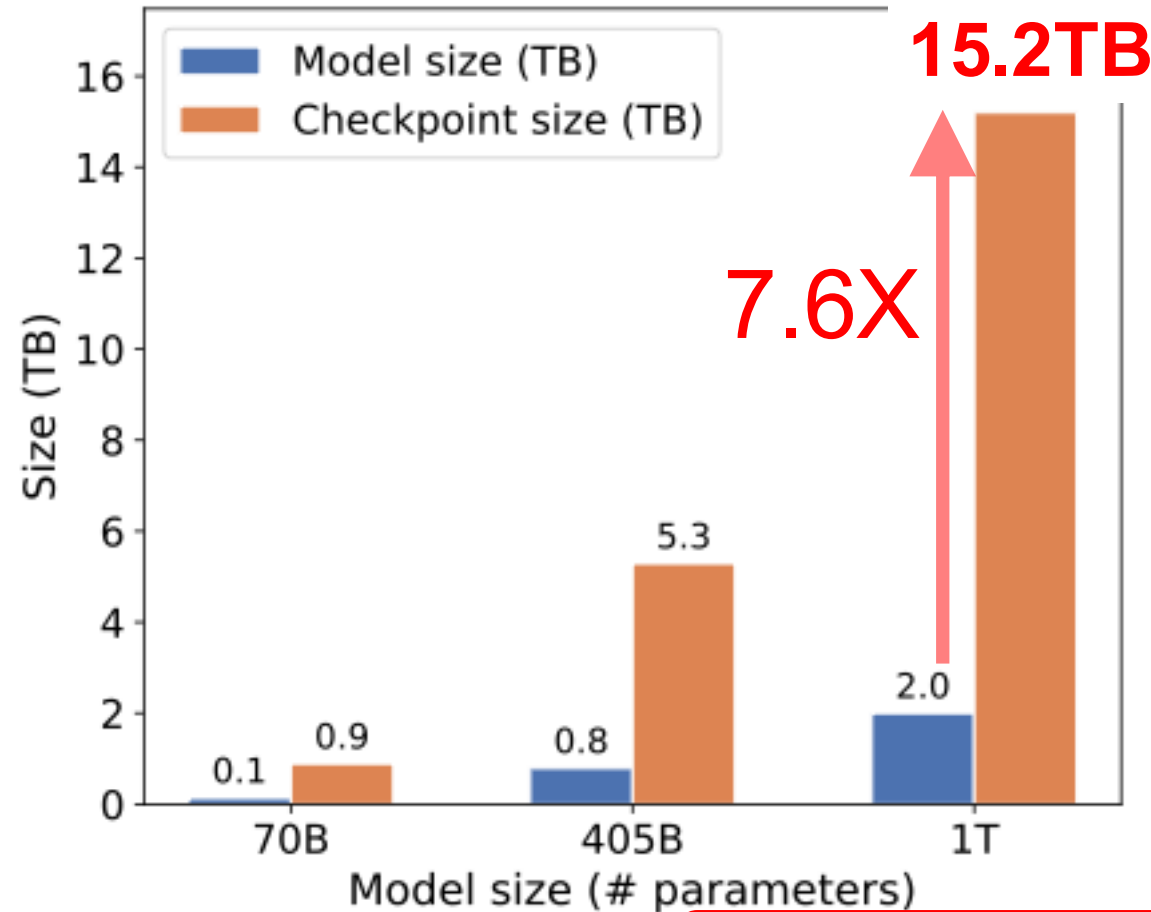
155 checkpoints/day

1T model
>2.3PB checkpoint writes/day

The Llama 3 Herd of Models: <https://arxiv.org/pdf/2407.21783>

MLCommons MLPerf Storage v2.0 Checkpoint Workload
<https://mlcommons.org/2025/08/storage-2-checkpointing/>

Recap: LLM training checkpoints



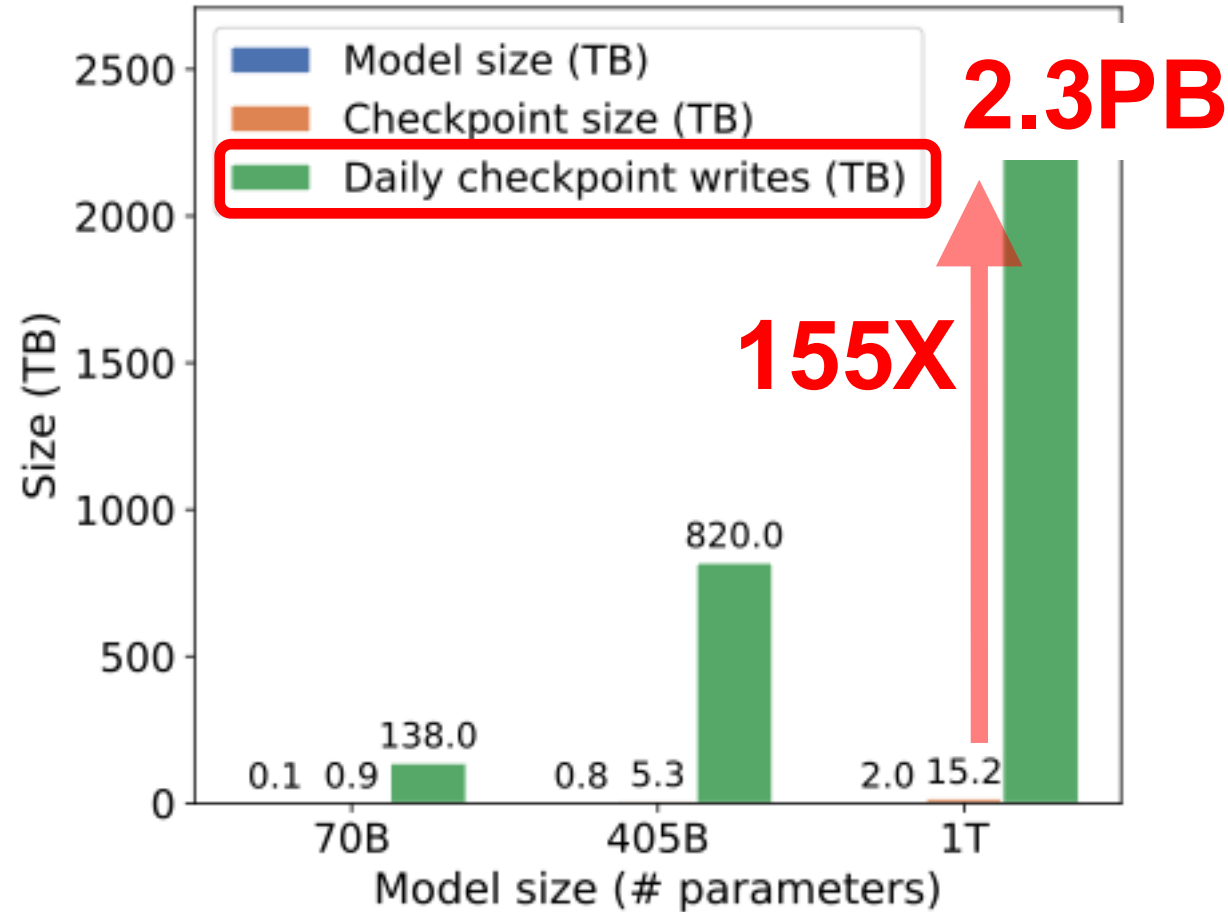
Checkpoint size = model weights + **optimizer state**

- Master weights (FP32)
- Momentum (FP32)
- Variance (FP32)d

*: <https://mlcommons.org/2025/08/storage-2-checkpointing/>

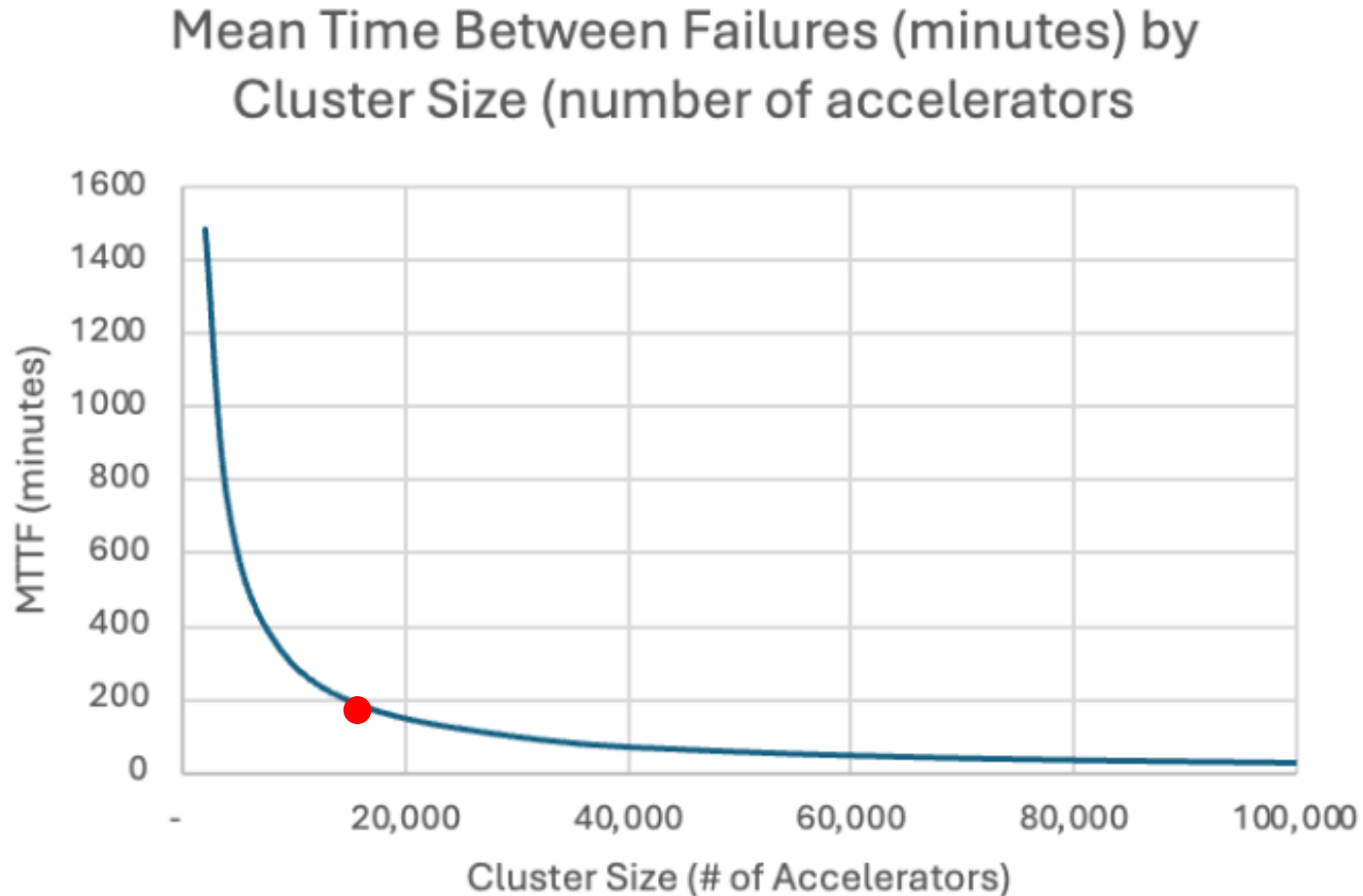
Recap: LLM training checkpoints

A 16,000-GPU cluster has a MTTF of 186 minutes



A trainer should checkpoint 150+ times a day to mitigate the high cost of frequent failures at 16,000-GPU scale

Recap: MTBF and training reliability

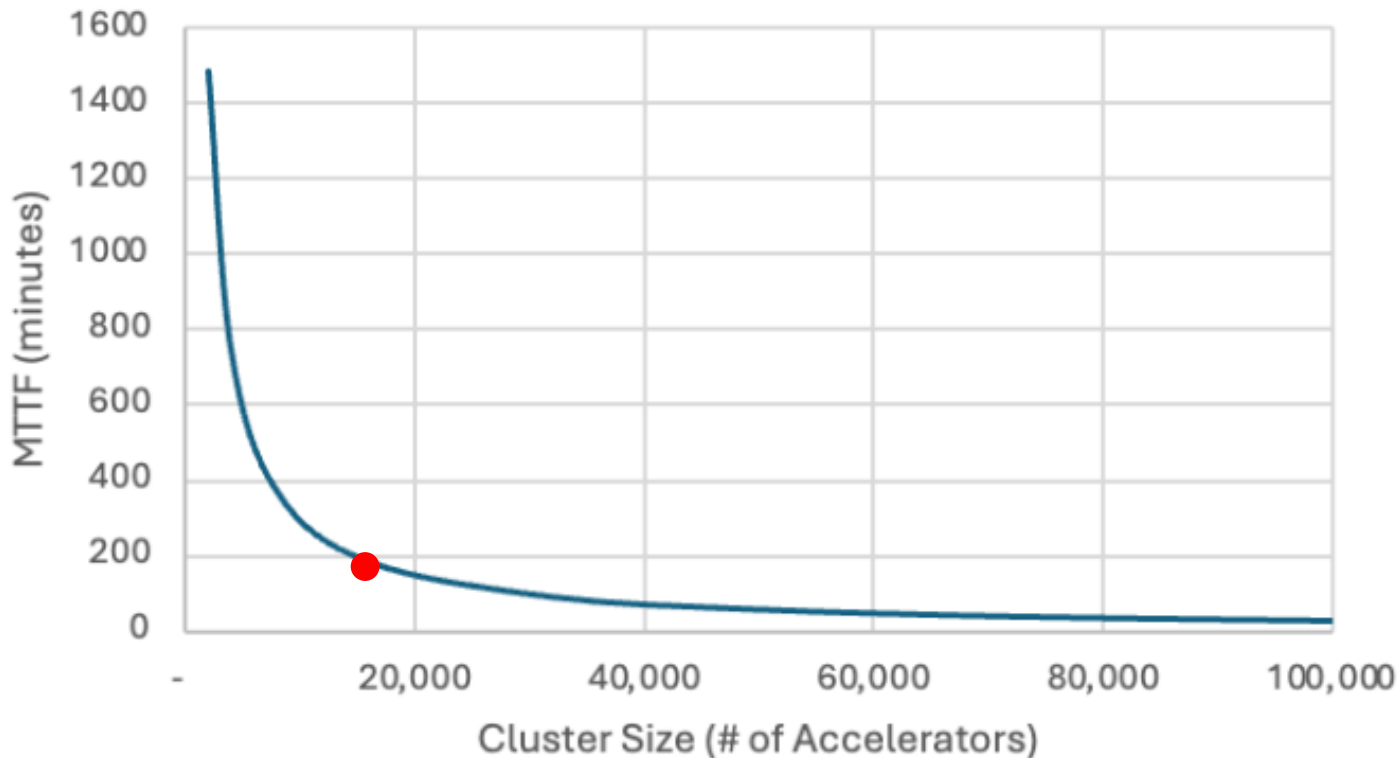


MLCommons MLPerf Storage v2.0 Checkpoint Workload
<https://mlcommons.org/2025/08/storage-2-checkpointing/>

The Llama 3 Herd of Models: <https://arxiv.org/pdf/2407.21783>

Recap: MTBF and training reliability

Mean Time Between Failures (minutes) by
Cluster Size (number of accelerators)



MLCommons MLPerf Storage v2.0 Checkpoint Workload
<https://mlcommons.org/2025/08/storage-2-checkpointing/>

The Llama 3 Herd of Models: <https://arxiv.org/pdf/2407.21783>

$$\tau_{\text{opt}} = \sqrt{2 \cdot T_{\text{write}} \cdot \text{MTBF}}$$

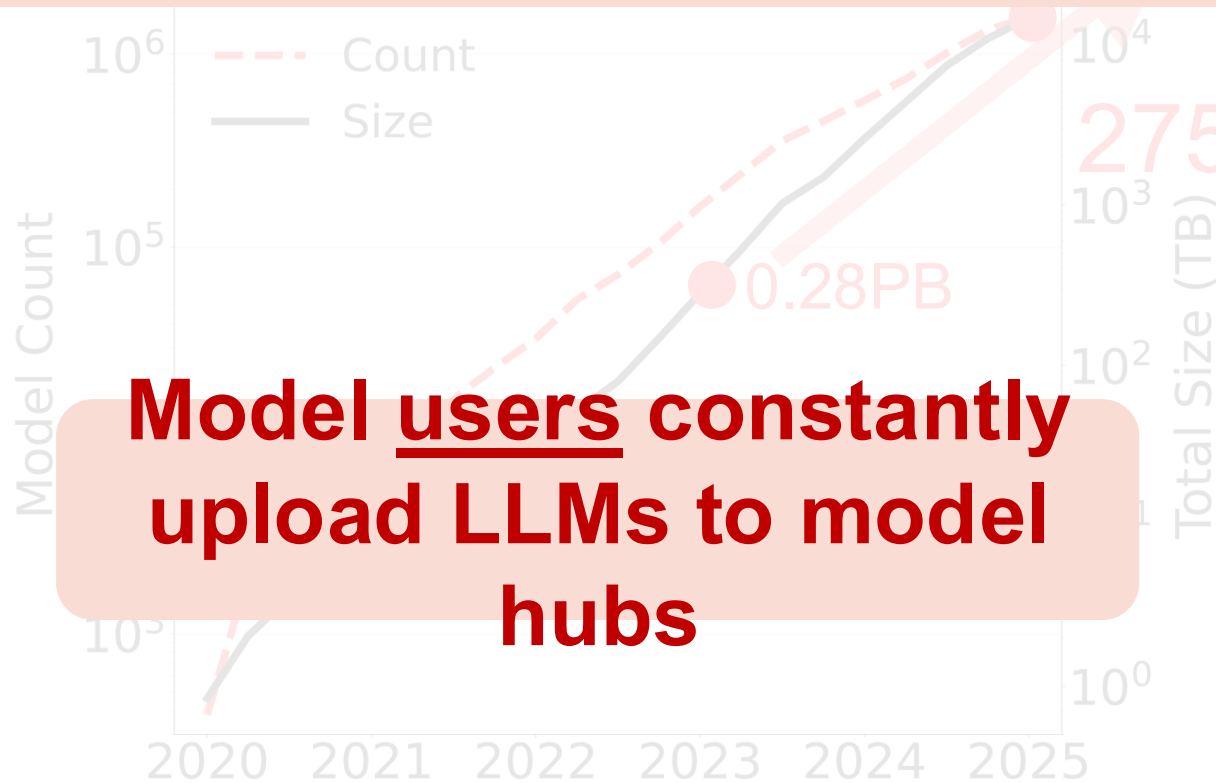
Assuming MTBF = 186 min

For a checkpoint size of 15.2 TB

Assumed ckpt write time T_{write}	Optimal checkpoint interval τ_{opt}
1 min	~19 min
2 min	~27 min
3 min	~33 min
4 min	~39 min
5 min	~43 min

The AI model storage crisis

Current storage demands are **unsustainable** for AI models



Model users constantly upload LLMs to model hubs



Hugging Face

Exponential growth

1EB model storage by 2027

Y. Cheng

1T model → **15TB** checkpoint

Meta

16K GPUs → **186 min MTE**

Model trainers frequently store massive checkpoints across steps/runs/versions

155 checkpoints/day **>2.3PB** checkpoint writes/day

The Llama 3 Herd of Models: <https://arxiv.org/pdf/2407.21783>

Source: MLCommons MLPerf Storage v2.0 Checkpoint Workload
<https://mlcommons.org/2025/08/storage-2-checkpointing/> 10

A large stack of cardboard boxes, many of which feature logos for various AI models and companies. The logos visible include OpenAI GPT-4o, DeepSeek, Anthropic Claude, Google Gemini, and others. The boxes are stacked in a way that creates a sense of depth and volume, with some boxes partially obscured by others. The logos are prominently displayed on the front of several boxes, making them easily identifiable.

Y. Cheng

What's hidden inside model storage?



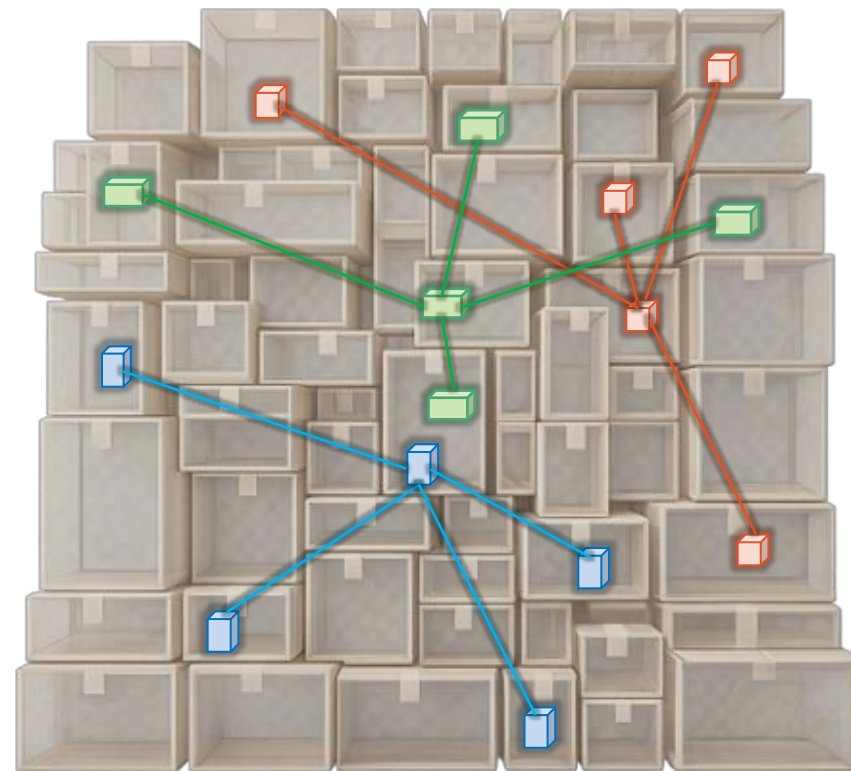
Hugging Face



Models stored as **independent**, black-box files



Models are (massive, messy) **datasets**
Hidden structure across models and tensors



Model-level relationships (lineage)
Tensor-level relationships (content, similarity)

Why does this matter?



Hugging Face

Hidden structure across models and tensors

Discovering & exploiting hidden structure

Rethinking modern AI infrastructure

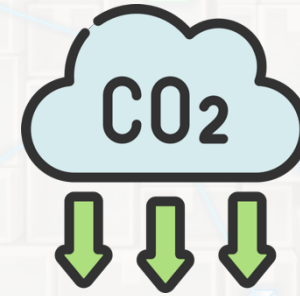
Reducing infrastructure cost, energy, and carbon at scale



Cost



Energy



Carbon

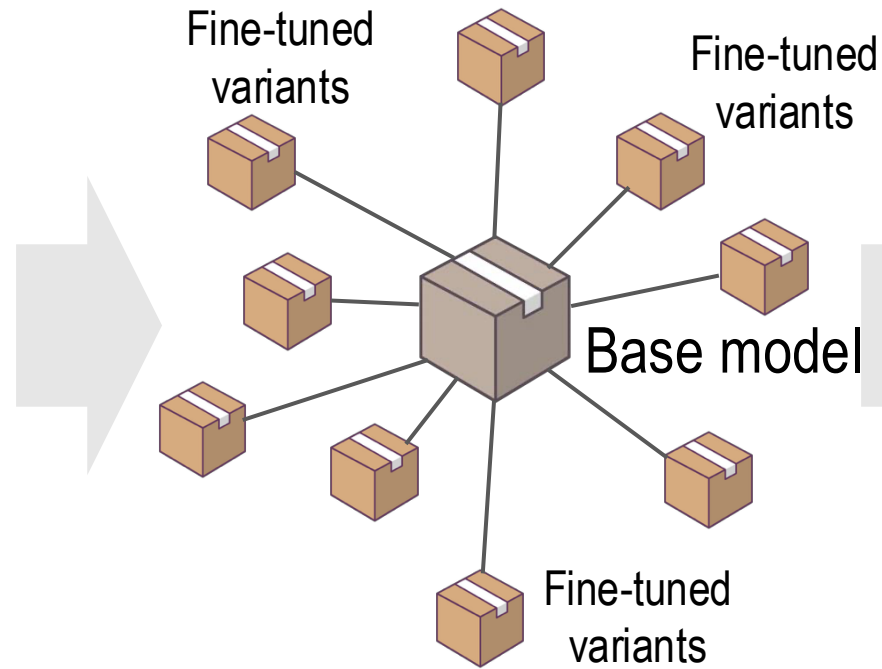
Models stored as **independent**, black-box files

Model-level relationships (lineage)
Tensor-level relationships (content, similarity)

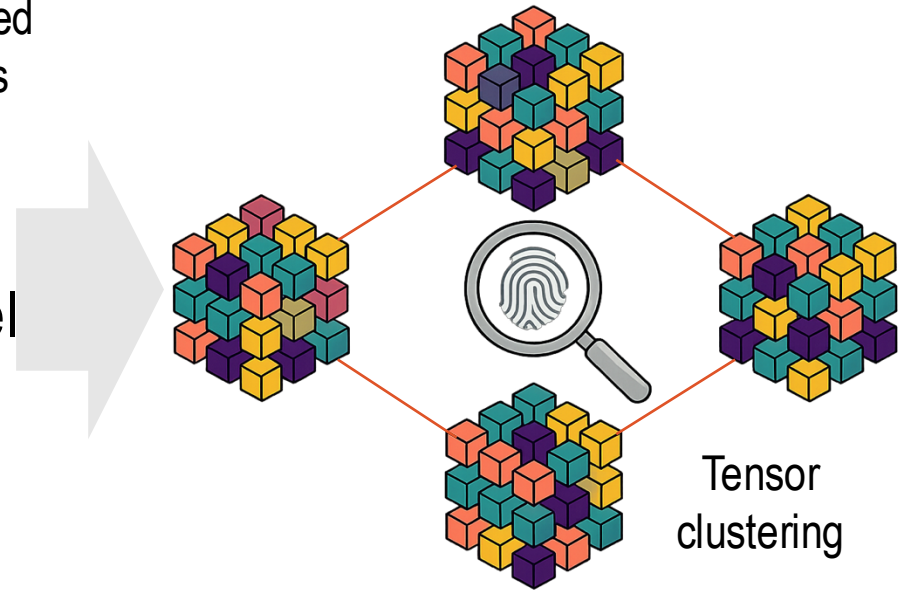
From ZipLLM to TensorDex



**Characterizing Hugging
Face model hub**



**ZipLLM:
Model-level compression**

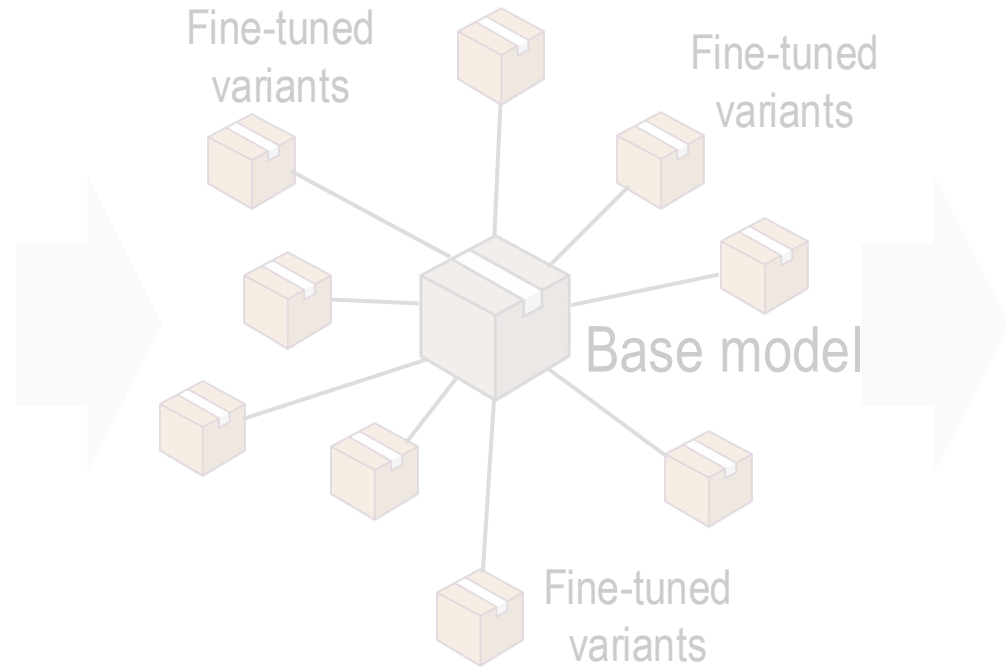


**TensorDex:
Tensor-level compression**

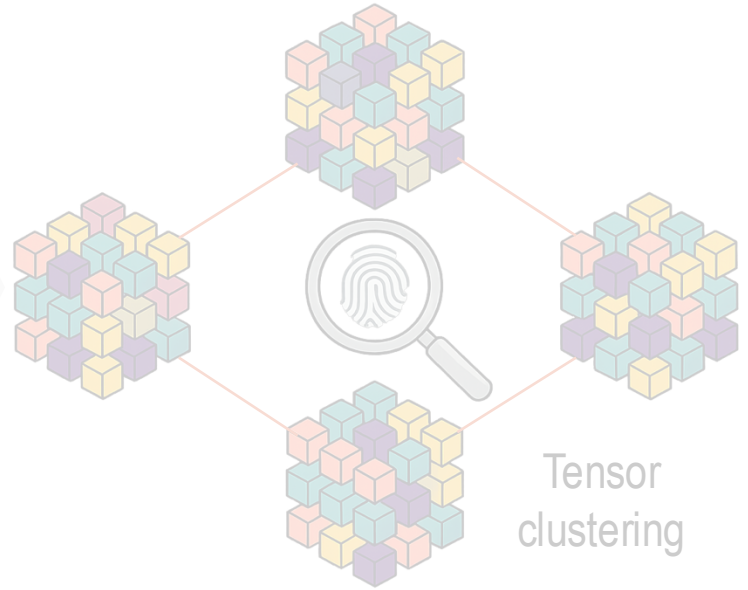
Outline



**Characterizing Hugging
Face model hub**



**ZipLLM:
Model-level compression**

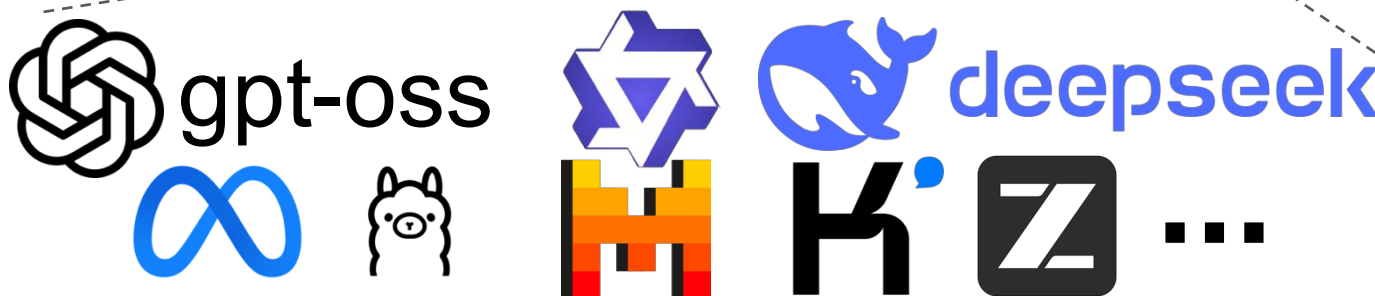
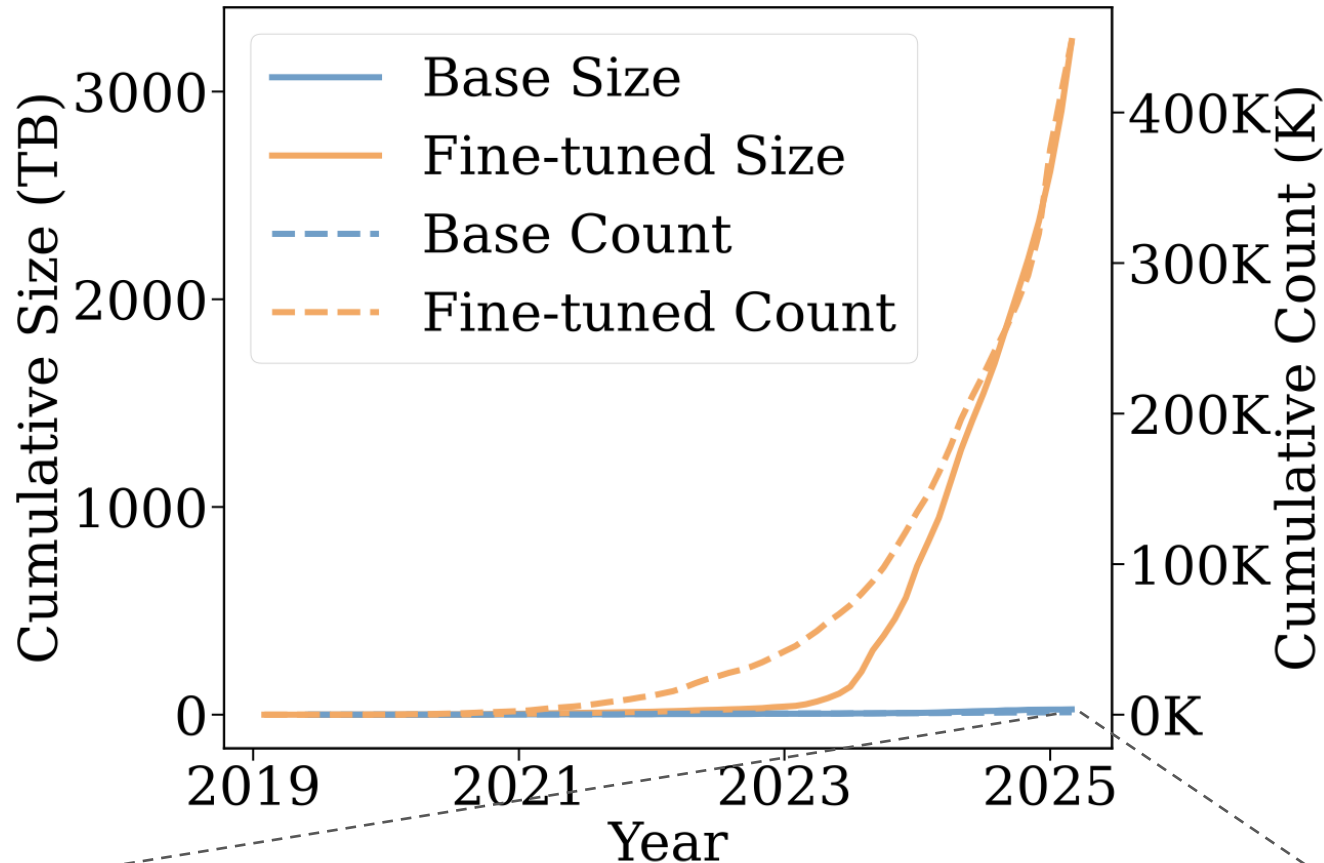


**TensorDex:
Tensor-level compression**

Characterizing model hub

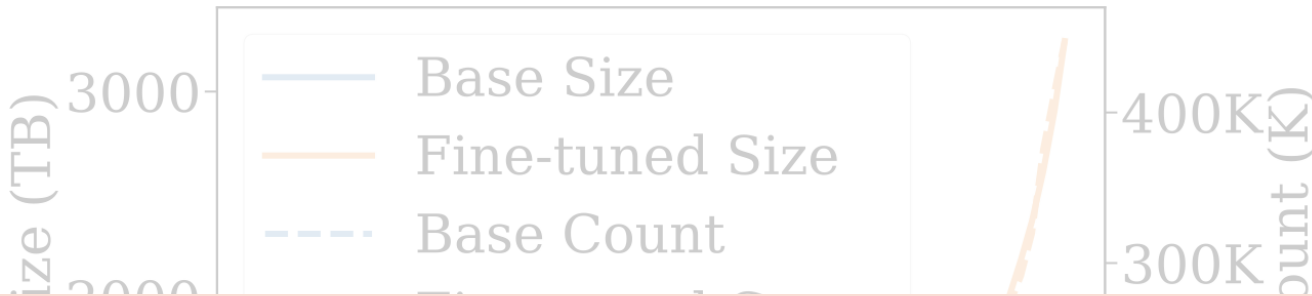
- Collected **all** LLM repos from Hugging Face
- Conducted the **first-of-its-kind**, large-scale field characterization study focusing on LLM storage
- **Observation 1:** **Fine-tuned** LLMs dominate
- **Observation 2:** **Fine-tuned** LLMs resemble their **base** models

Observation 1: Fine-tuned LLMs dominate



- Rise of open-weight LLMs
- Only **30+** open-weight LLMs widely used for fine-tuning!
- Fine-tuned LLMs are growing much faster than base models
- Storage consumption is overwhelmingly dominated by fine-tuned LLMs

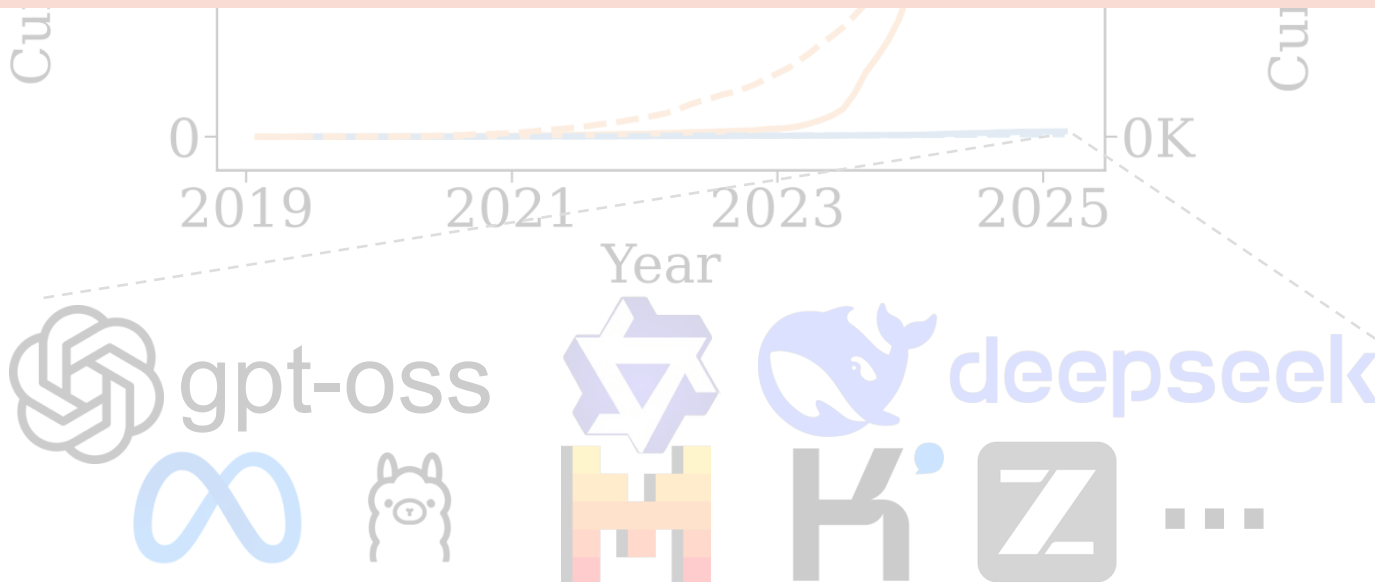
Observation 1: Fine-tuned LLMs dominate



- Rise of open-weight LLMs

- Only **30+** open-weight LLMs widely used for fine-tuning

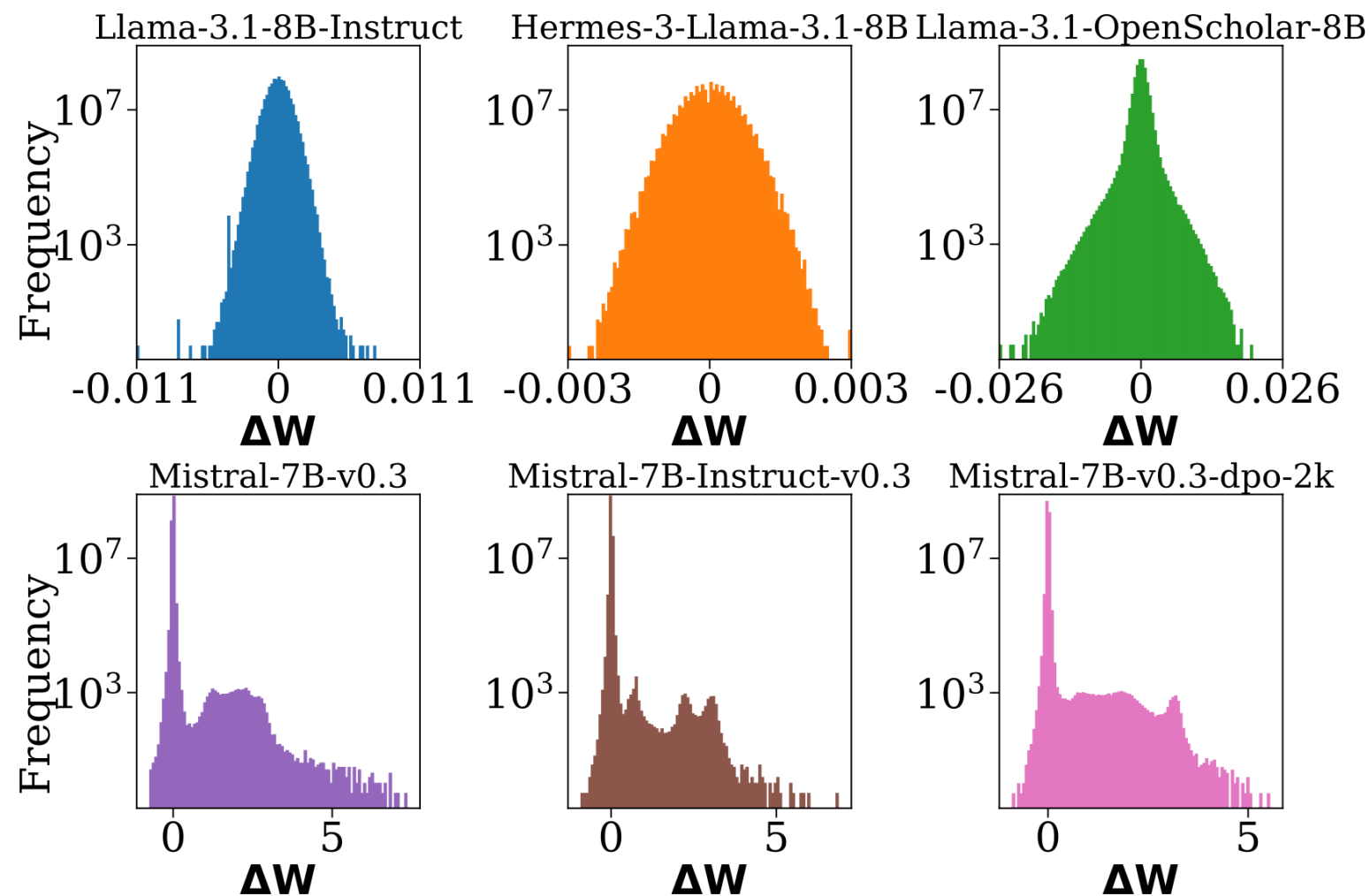
Finding 1: Fine-tuned LLMs dominate storage cost—optimization should focus here



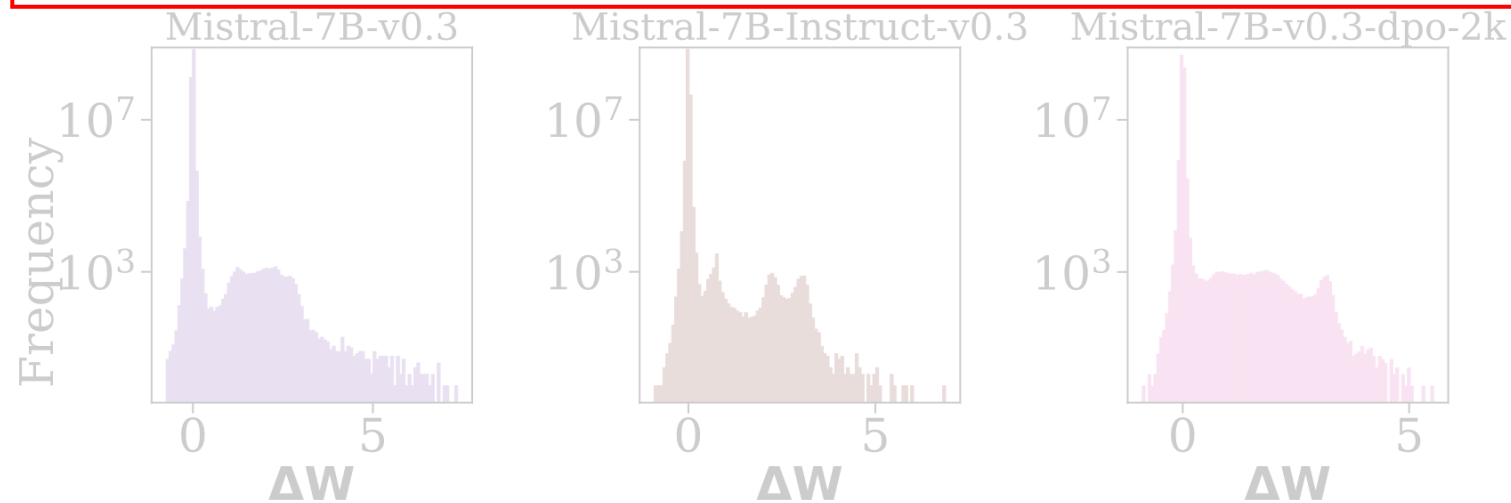
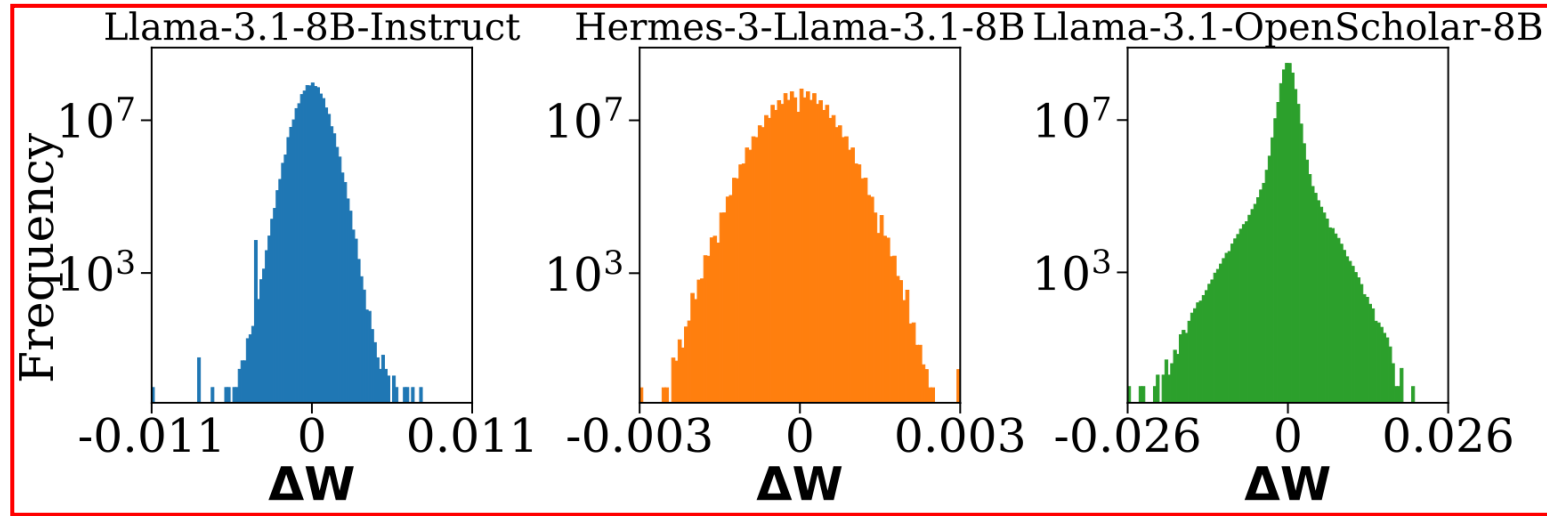
growing much faster than base models

- Storage consumption is overwhelmingly dominated by fine-tuned LLMs

Observation 2: Fine-tuned LLMs resemble base models



Observation 2: Fine-tuned LLMs resemble base models



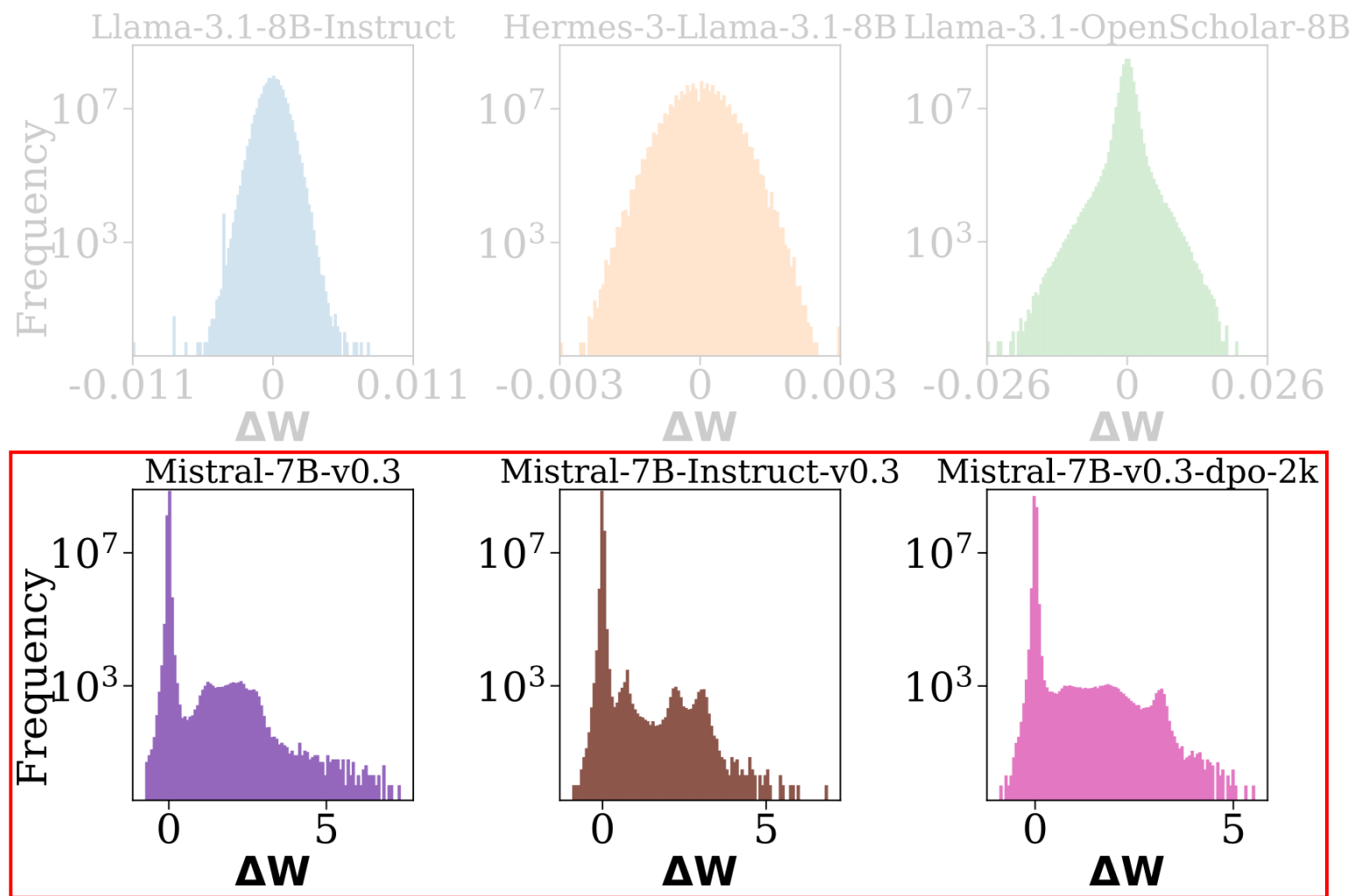
Fine-tuned from Llama-3.1-8B

$$\leftarrow \Delta w_i = w_i - \widehat{w}_i$$

Fine-tuned model weight Base model weight

- Weight deltas Δw_i are centered around 0
- Majority of base LLM weights **unchanged** in fine-tuned variants

Observation 2: Fine-tuned LLMs resemble base models



- Weight deltas from different LLM families are much wider

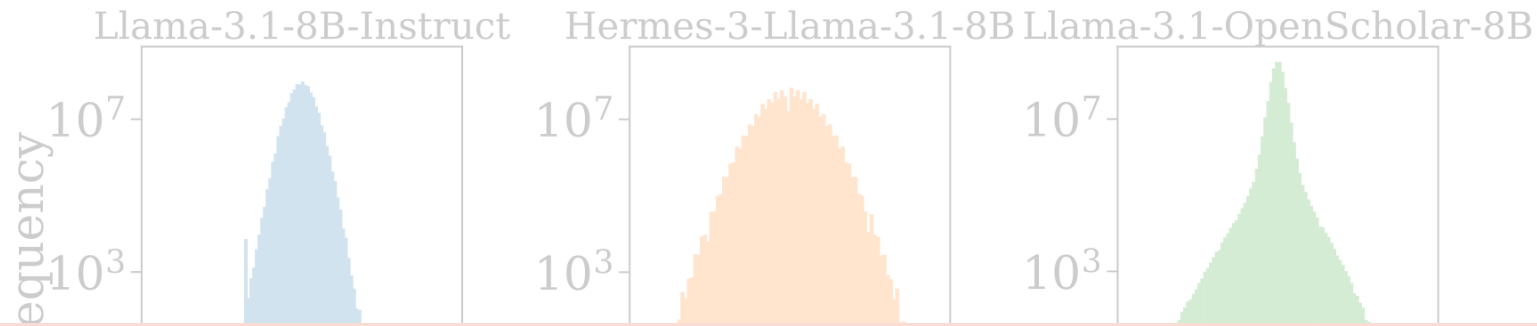


Fine-tuned from Mistral-7B-v0.3

$$\Delta w_i = w_i - \widehat{w}_i$$

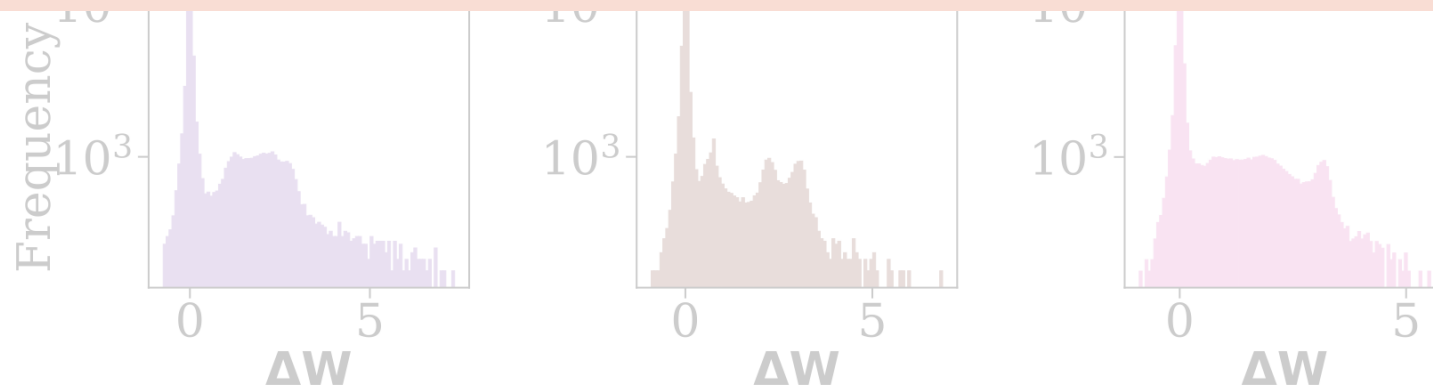
← Fine-tuned Mistral-7B-v0.3 model weight Llama-3.1-8B base model weight

Observation 2: Fine-tuned LLMs resemble base models



- Weight deltas from different LLM families are

Finding 2: LLM variants within a family are numerically similar and thus can benefit from **delta compression**



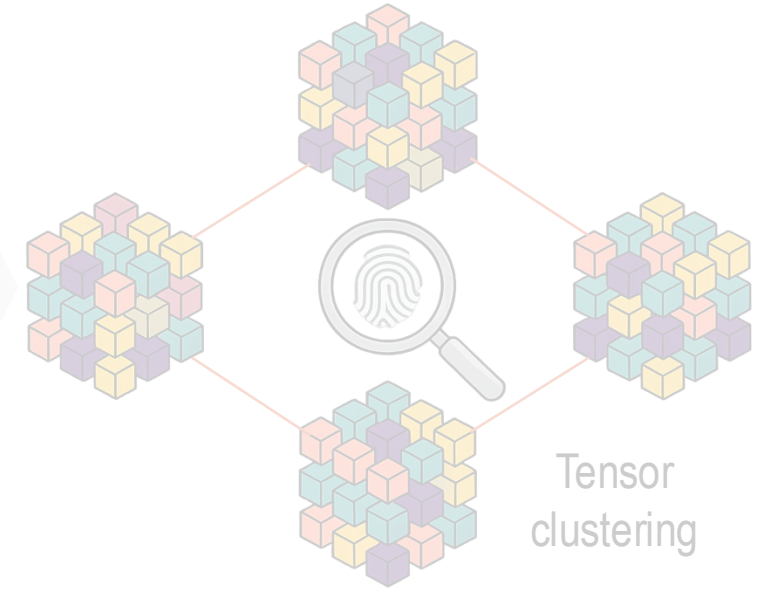
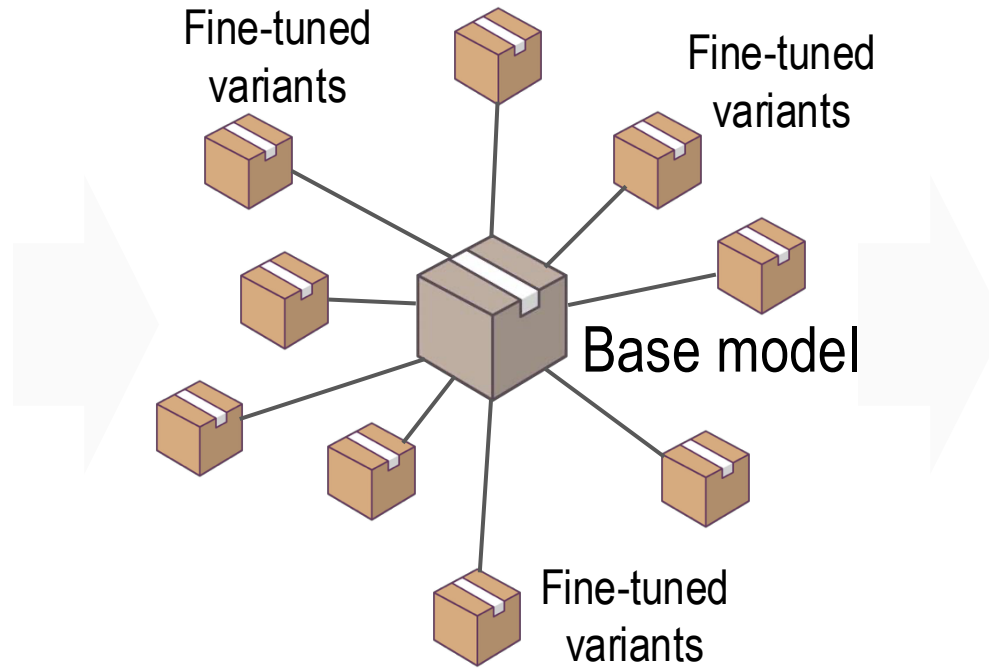
$$\Delta w_i = w_i - \hat{w}_i$$

←

Fine-tuned **Mistral-7B-v0.3** model weight

Llama-3.1-8B base model weight

Talk outline

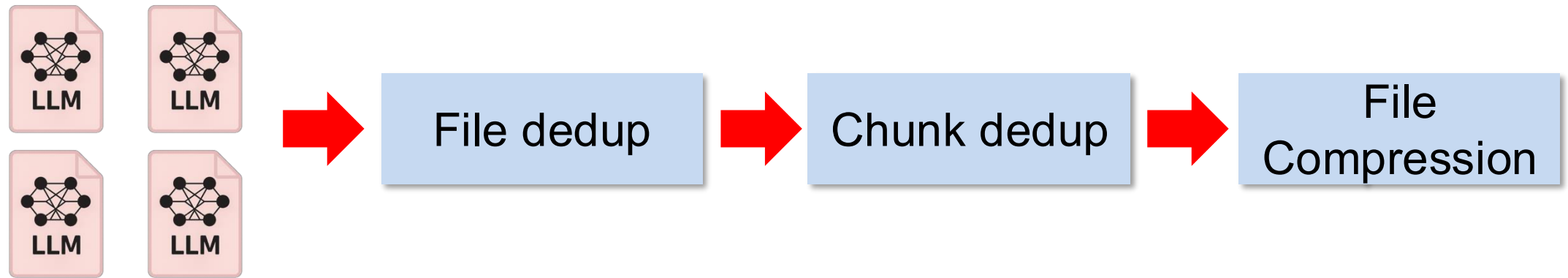


Characterizing Hugging
Face model hub

ZipLLM:
Model-level compression

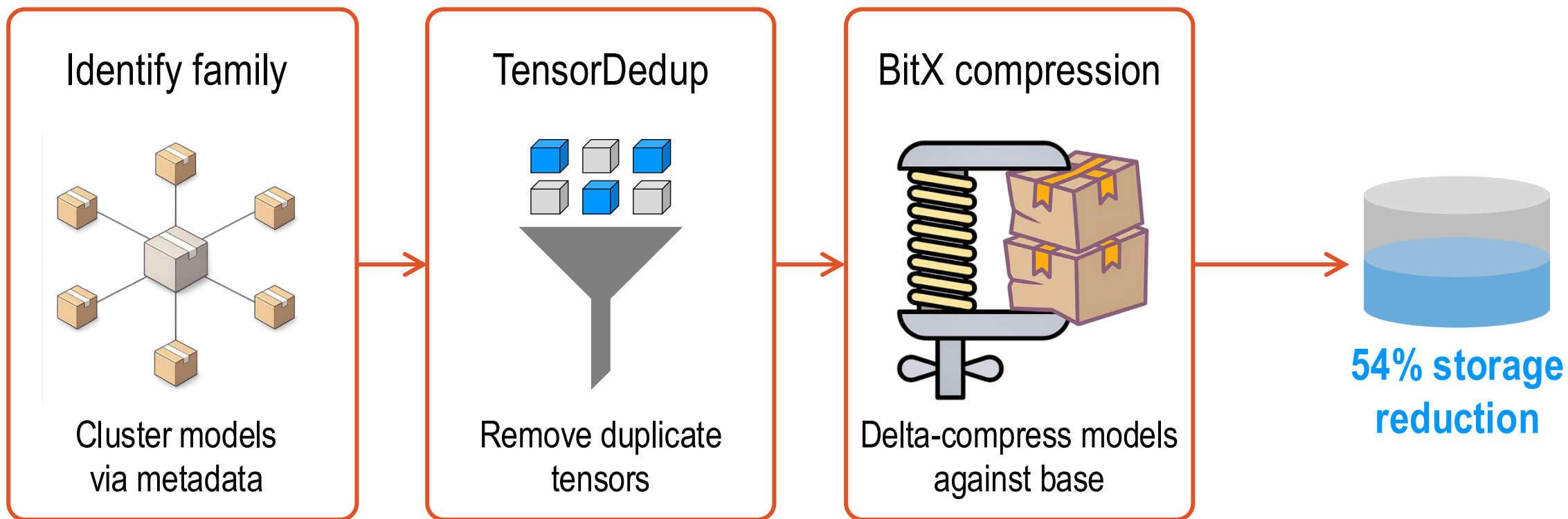
TensorDex:
Tensor-level compression

Hugging Face's storage reduction pipeline

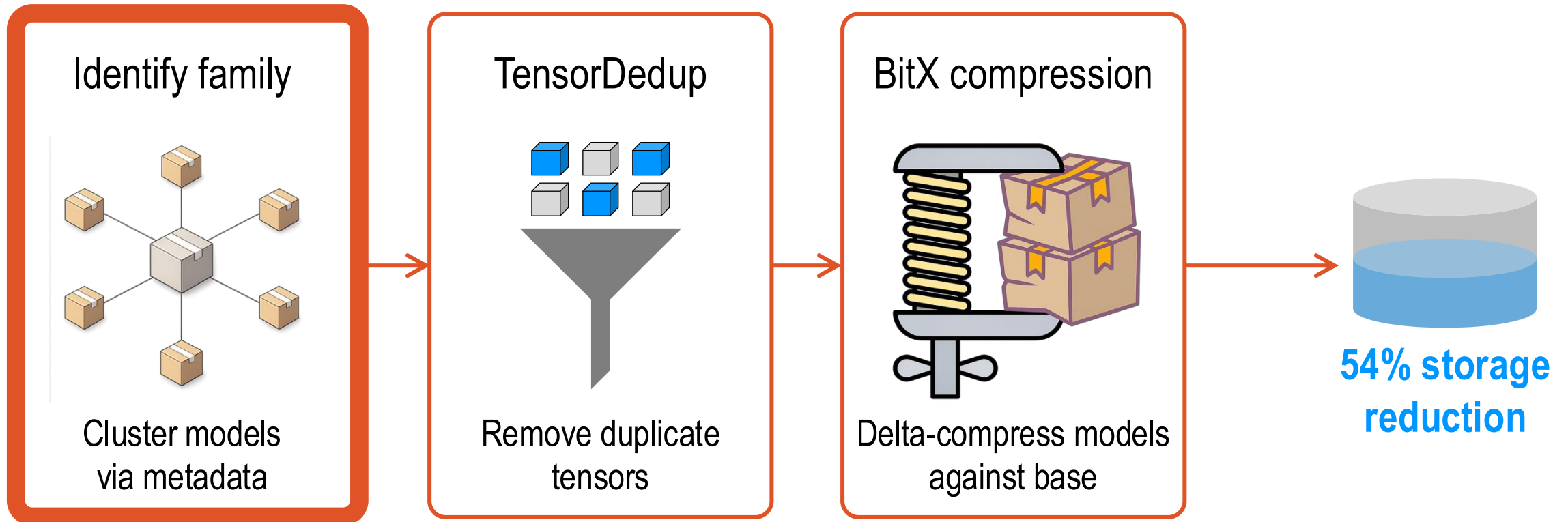


- **File dedup:** Removes identical model files across repos
- **Chunk dedup** (content-defined chunking): Splits files into variable-sized chunks by content, detects and removes duplicate chunks
- **File compression:** Applies lossless compression (e.g., zstd, ZipNN*) on remaining data

ZipLLM redesigns model storage reduction pipeline

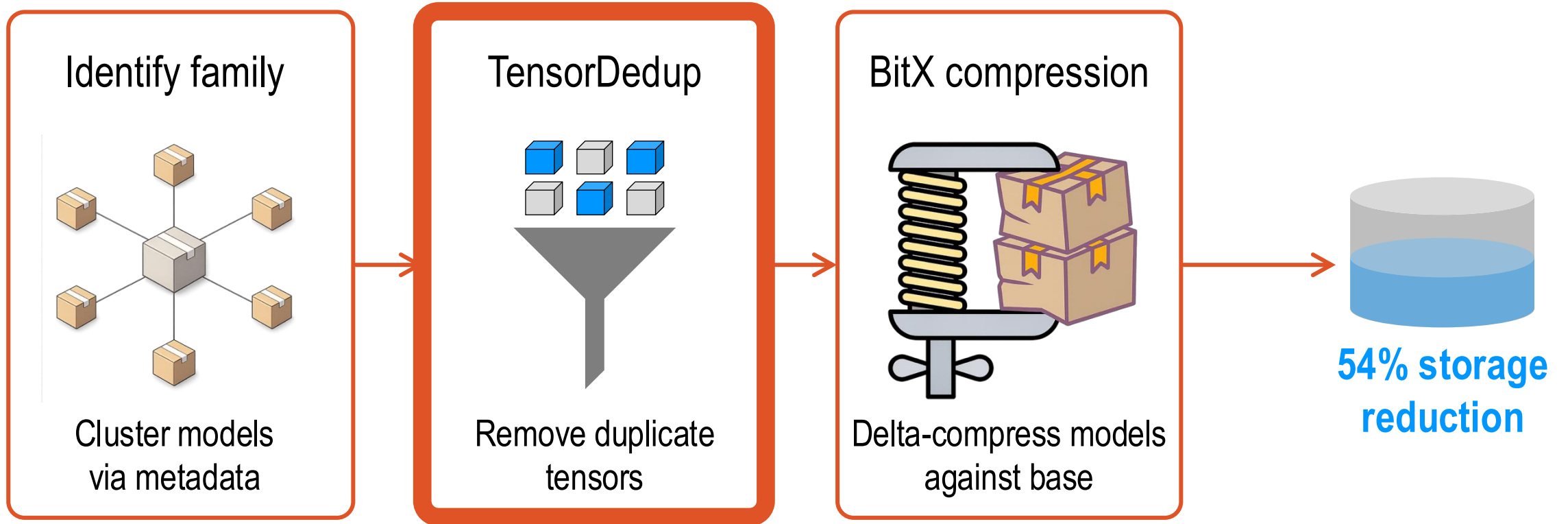


Phase 1: Identify model lineage information



Identify LLM lineage with user-provided metadata (model card) or content-aware metrics (bit distance)

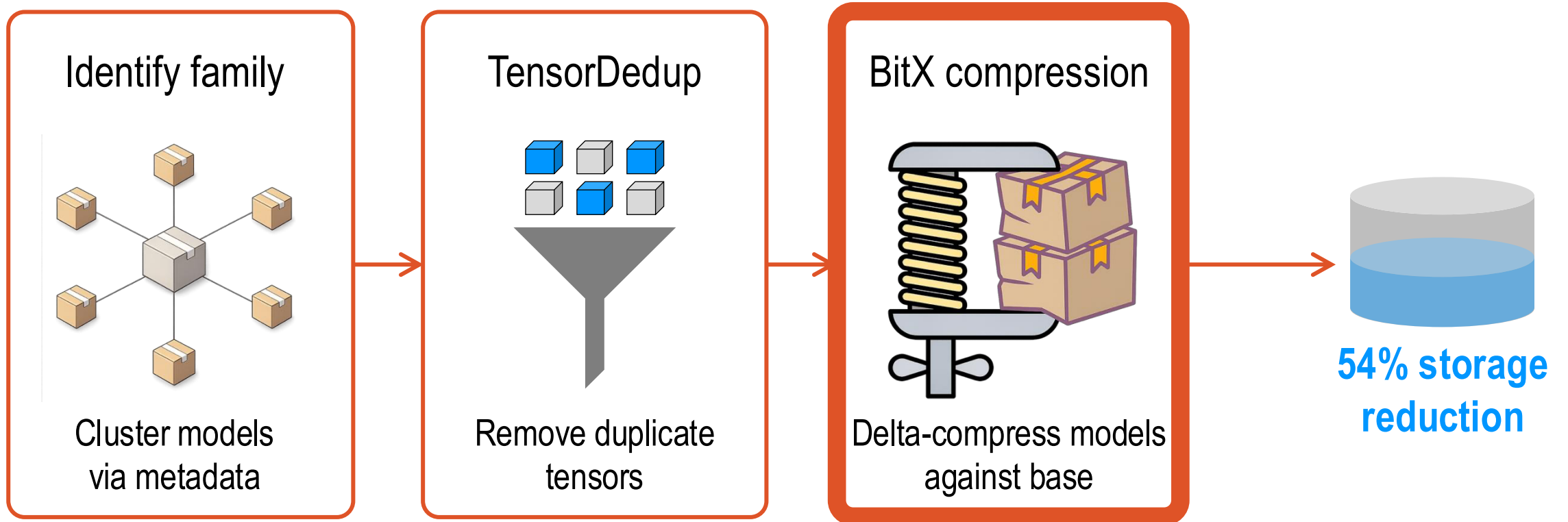
Phase 2: TensorDedup



Leverages safetensors format for efficient tensor dedup

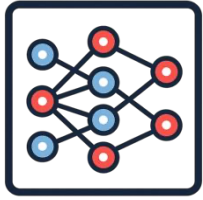
- Significantly reduced dedup index metadata size → **1,000X** smaller indices than chunk-based dedup
- Tensor-level **parallel dedup** → Higher ingestion throughput

Phase 3: BitX delta compression

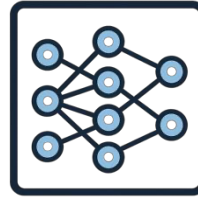


BitX: Model-aware delta compression

Base LLM



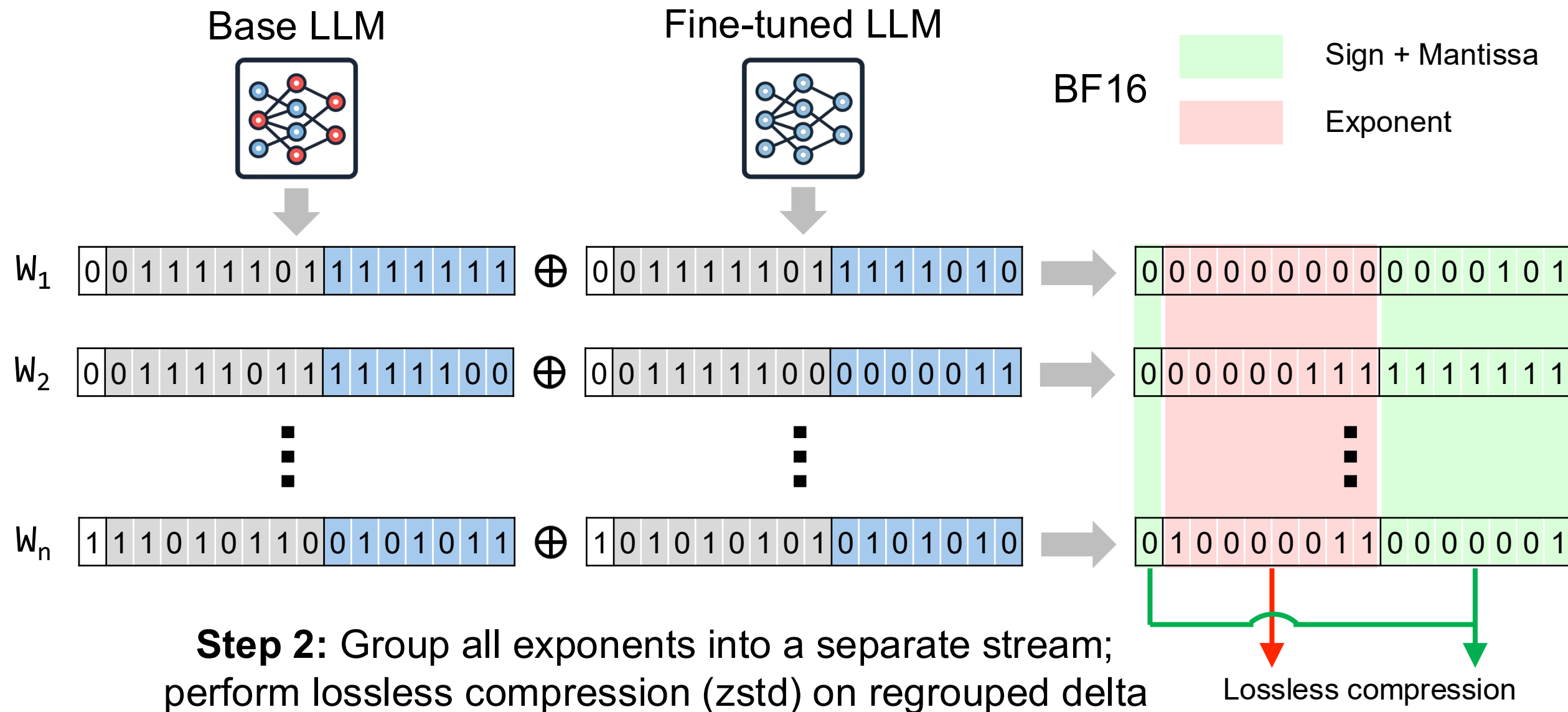
Fine-tuned LLM



$$\begin{array}{l} W_1 \quad \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ \hline \end{array} \oplus \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ \hline \end{array} \\ W_2 \quad \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ \hline \end{array} \oplus \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ \hline \end{array} \\ \vdots \\ W_n \quad \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ \hline \end{array} \oplus \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ \hline \end{array} \end{array}$$

Step 1: Perform XOR between base and fine-tuned LLM

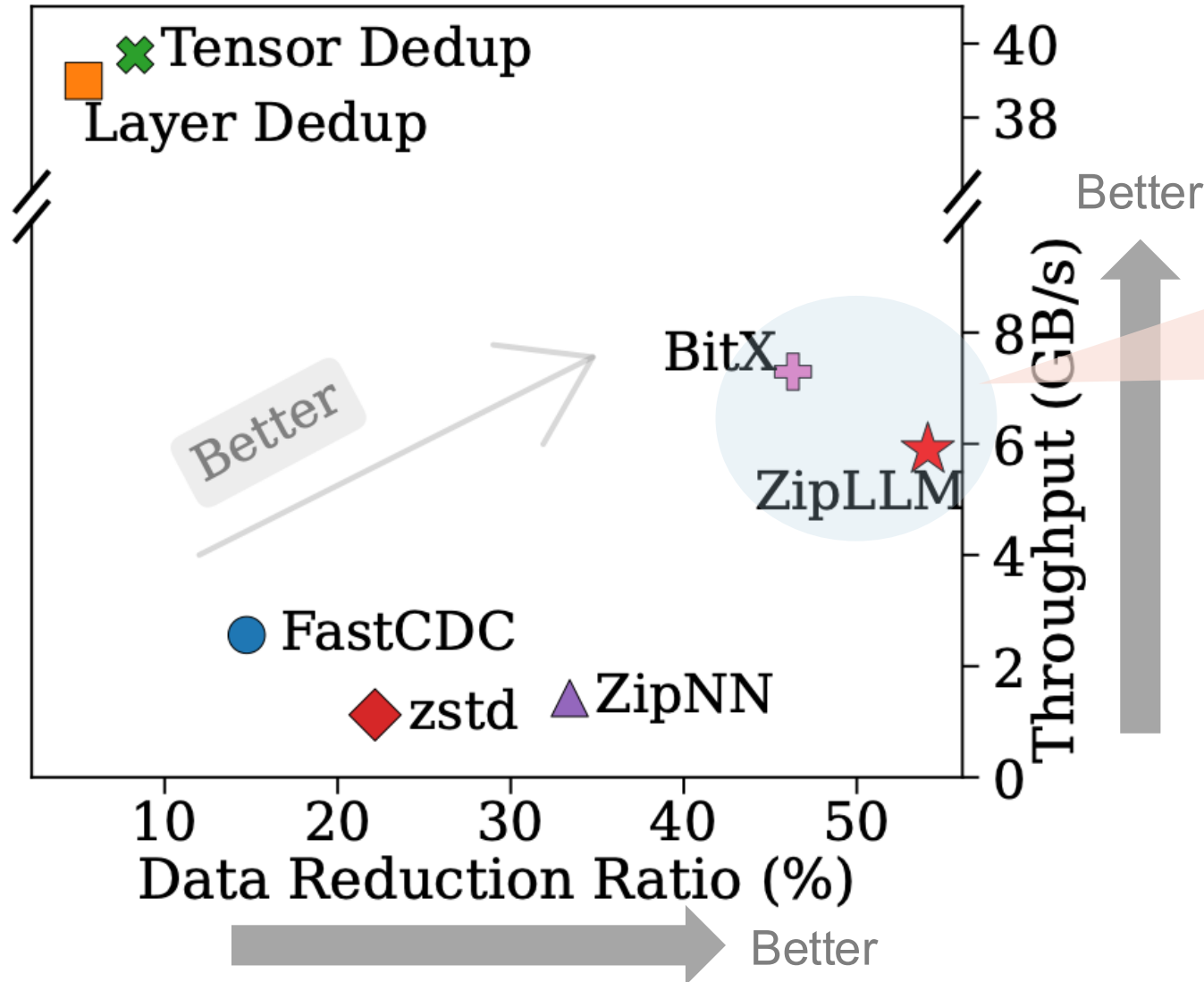
BitX: Model-aware delta compression



Evaluation setup

- **Dataset:** 3,000+ randomly sampled Hugging Face LLM repos, ~43 TB of **safetensors**
 - 968 Qwen2.5 models, 151 Qwen3 models, 139 Mistral models, 114 Llama-3 models, 1,431 Llama-3.1 models, 47 Llama-3.2 models, 135 Gemma-2 models, and 63 Gemma-3 models
- **Environment:** An EC2 c6a.48xlarge w/ 96 cores + 384GB DRAM

Data reduction ratio vs. throughput



BitX and ZipLLM
consistently outperform all
popular and SOTA data
reduction methods

Open-source, live gain, and impact

 Rust crate: [crates/bitx-codec](https://crates.io/crates/bitx-codec)

 GitHub: <https://github.com/ds2-lab/ZipLLM>

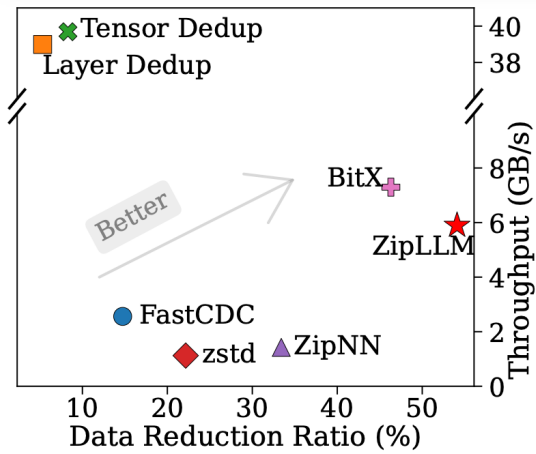
 Web: <https://ds2-lab.github.io/ZipLLM>

```
(base) ubuntu@ip-172-31-30-195:~/zipnn/scripts$ python3 zipnn_compress_file.py /home/ubuntu/code/ZipLLM/model1.safetensors --threads 192
```

ZipNN

```
(base) ubuntu@ip-172-31-30-195:~$ bitx compress /home/ubuntu/code/ZipLLM/model1.safetensors /home/ubuntu/code/ZipLLM/model2.safetensors -o diff.bitx
```

BitX



Published at USENIX NSDI 2026

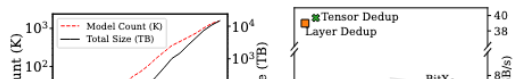
ZipLLM: Efficient LLM Storage via Model-Aware Synergistic Data Deduplication and Compression

Zirui Wang¹, Tingfeng Lan¹, Zhaoyuan Su¹, Juncheng Yang², Yue Cheng¹
¹University of Virginia, ²Harvard University



Abstract

Modern model hubs, such as Hugging Face, store tens of petabytes of LLMs, with fine-tuned variants vastly outnumbering base models. This paper introduces ZipLLM, a model-aware synergistic data deduplication and compression framework. ZipLLM achieves significant data reduction and throughput improvements over existing methods. For example, ZipLLM reduces the size of a 7B LLM by 40% while maintaining high throughput.



Collaborating with HF
toward production
integration

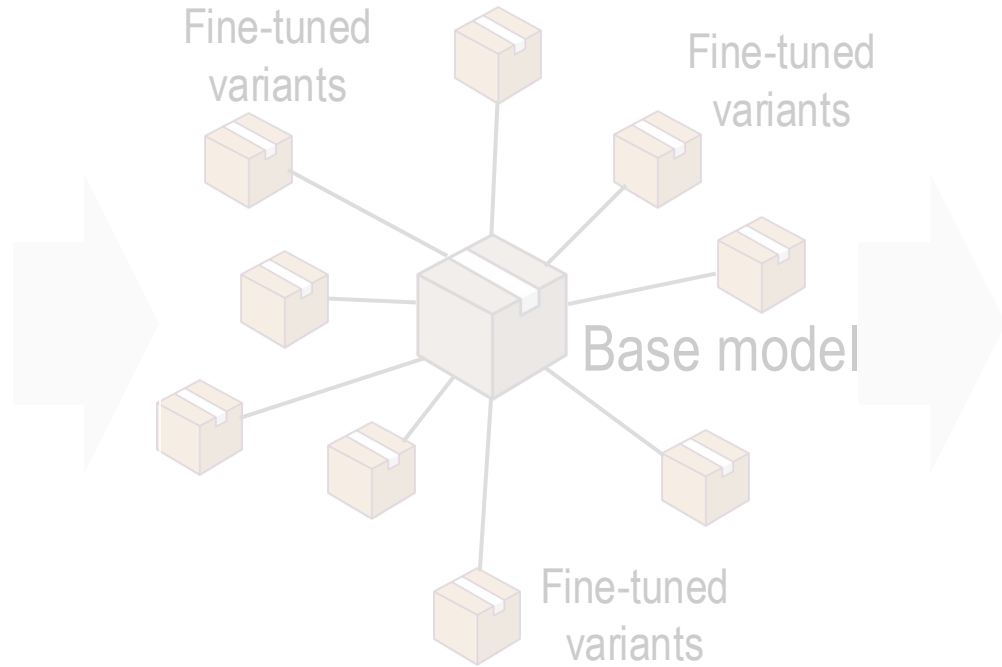


Hugging Face

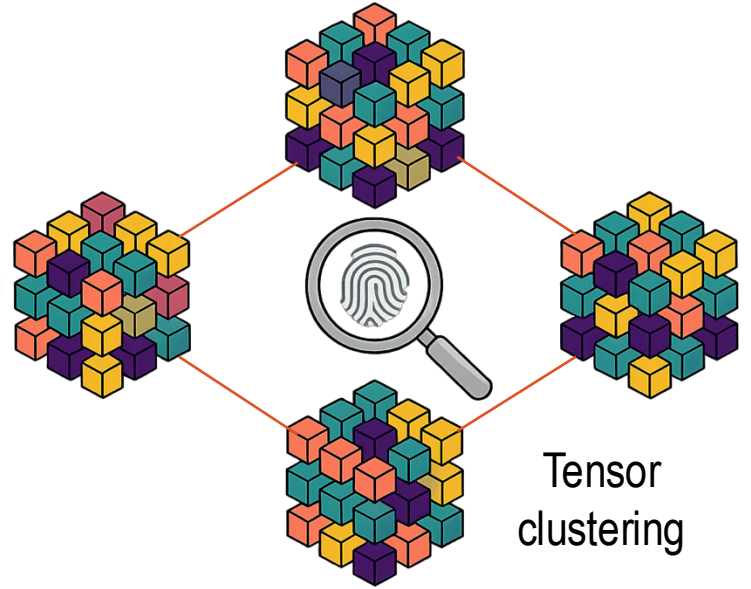
Outline



Characterizing Hugging
Face model hub



ZipLLM:
Model-level compression

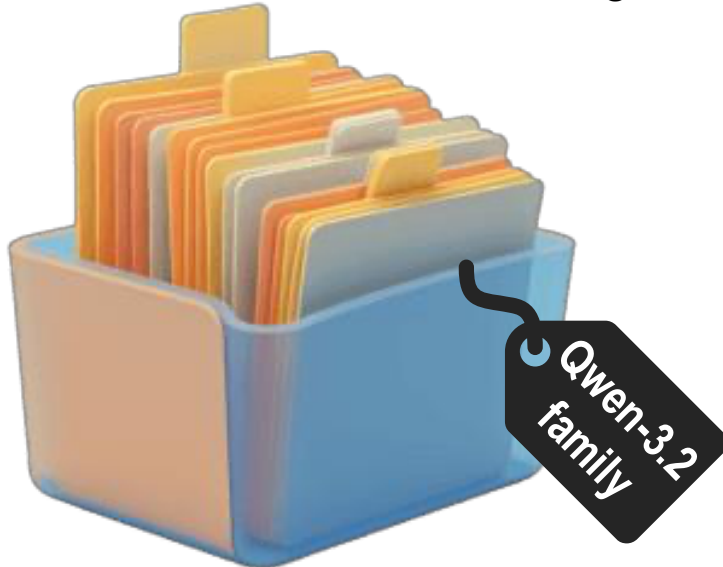


TensorDex:
Tensor-level compression

Problem 1: Real-world metadata fails

The Assumption

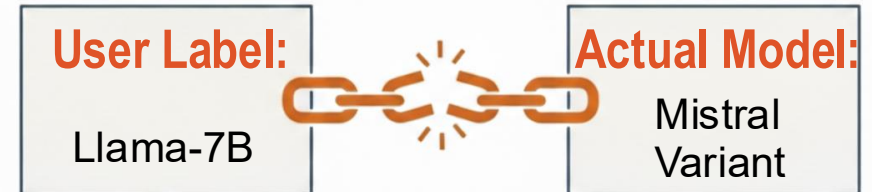
ZipLLM assumes accurate lineage



Requires correct "Base Model" tags

The Reality

Real-world repositories are messy

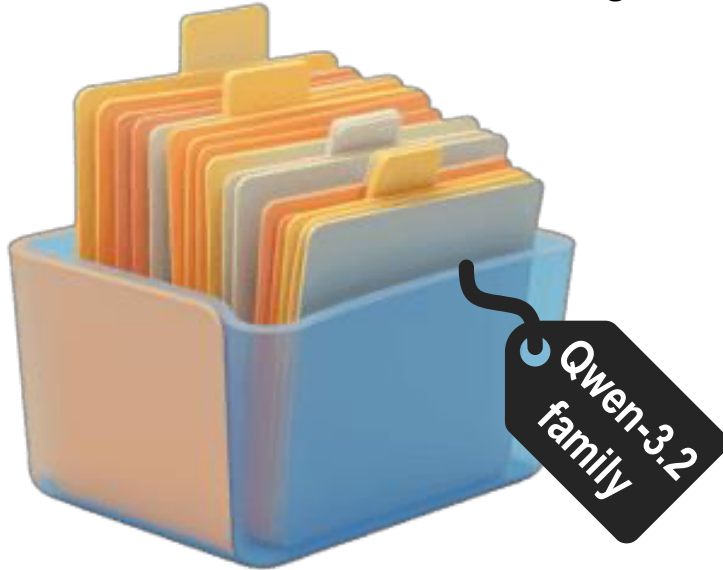


! Only **25.8%** out of 2.7M models have usable lineage metadata

Problem 1: Real-world metadata fails

The Assumption

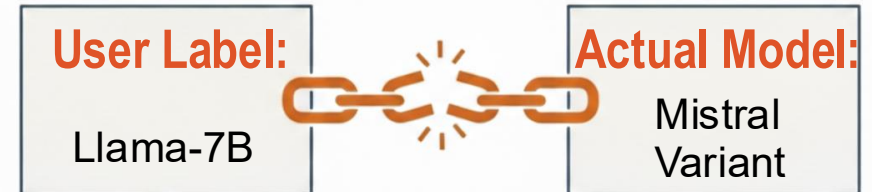
ZipLLM assumes accurate lineage



Requires correct "Base Model" tags

The Reality

Real-world repositories are messy



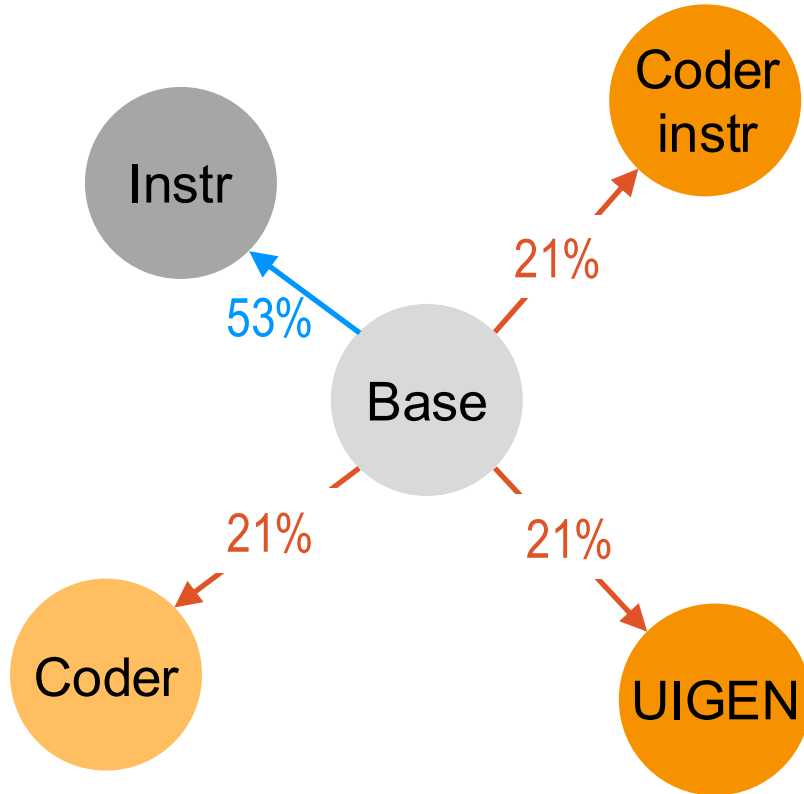
! Only **25.8%** out of 2.7M models have usable lineage metadata

Finding 3: User-provided metadata is missing/unreliable, making data-driven, metadata-oblivious pairing a necessity

Problem 2: Single-base assumption fails

The Assumption

ZipLLM assumes single-base compression

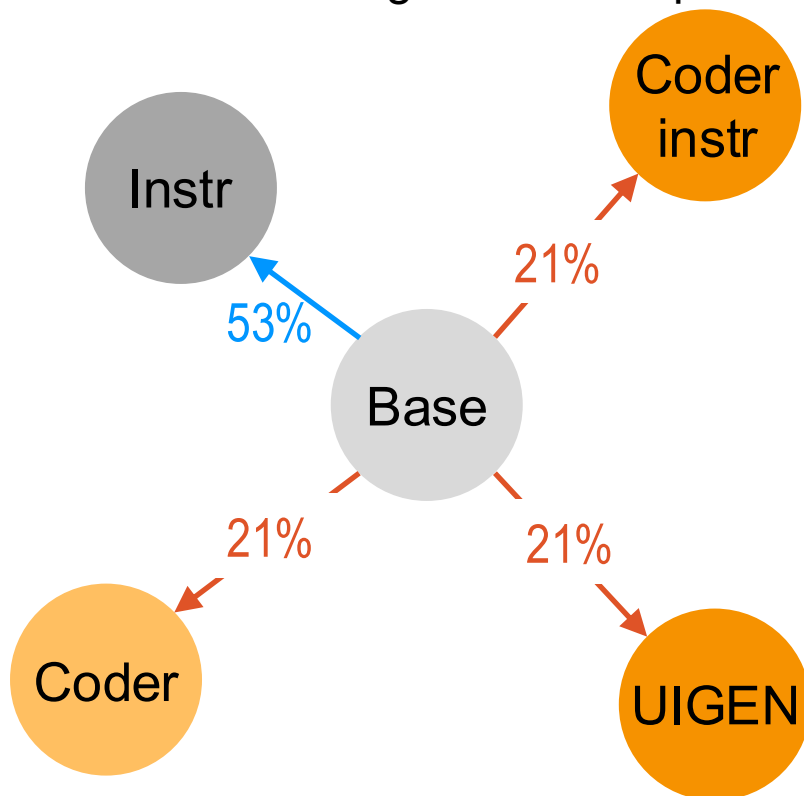


Overall reduction ratio: **29%**

Problem 2: Single-base assumption fails

The Assumption

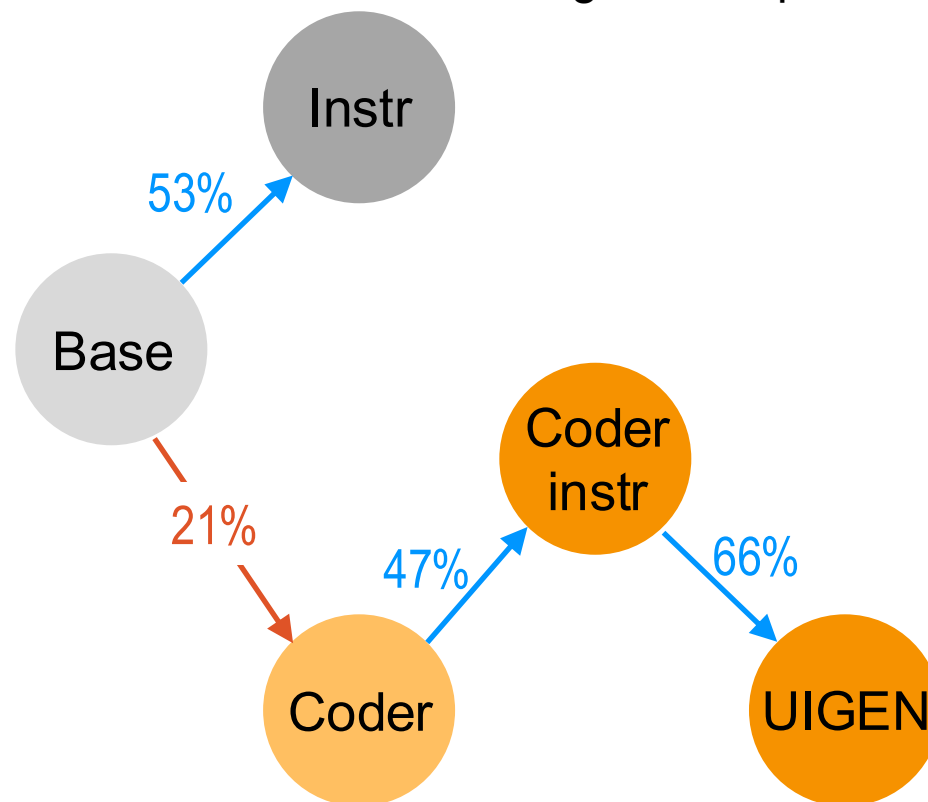
ZipLLM assumes single-base compression



Overall reduction ratio: **29%**

The Reality

Real-world LLM lineage is complex

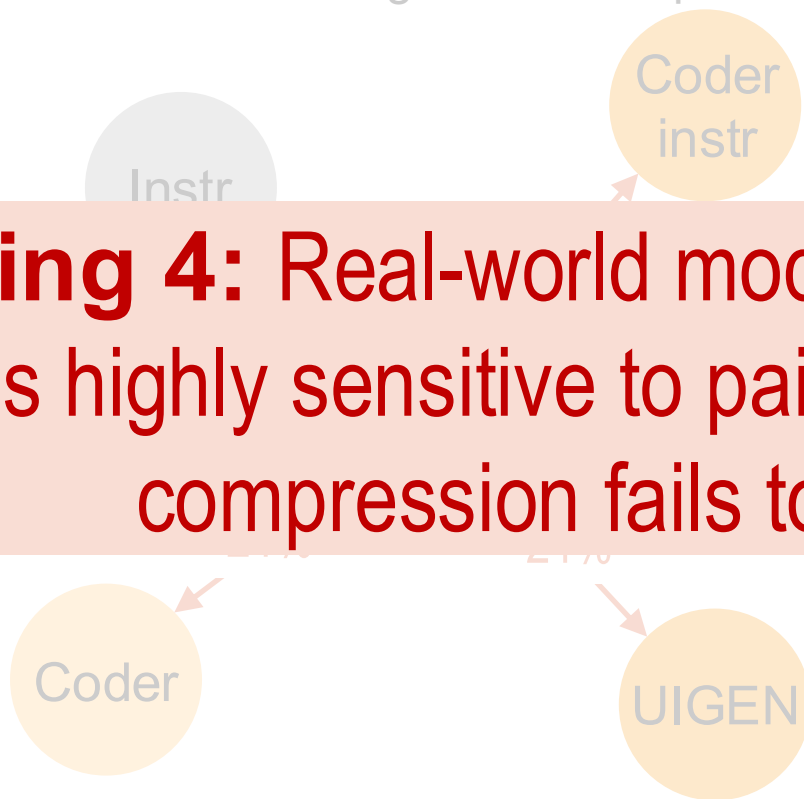


Overall reduction ratio: **47%**

Problem 2: Single-base assumption fails

The Assumption

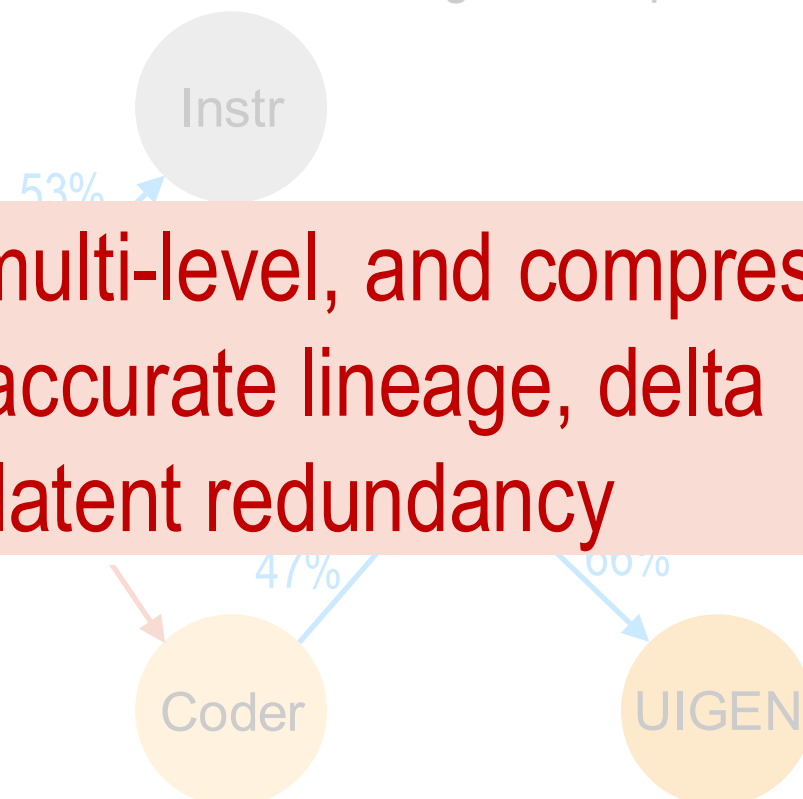
ZipLLM assumes single-base compression



Overall reduction ratio: 29%

The Reality

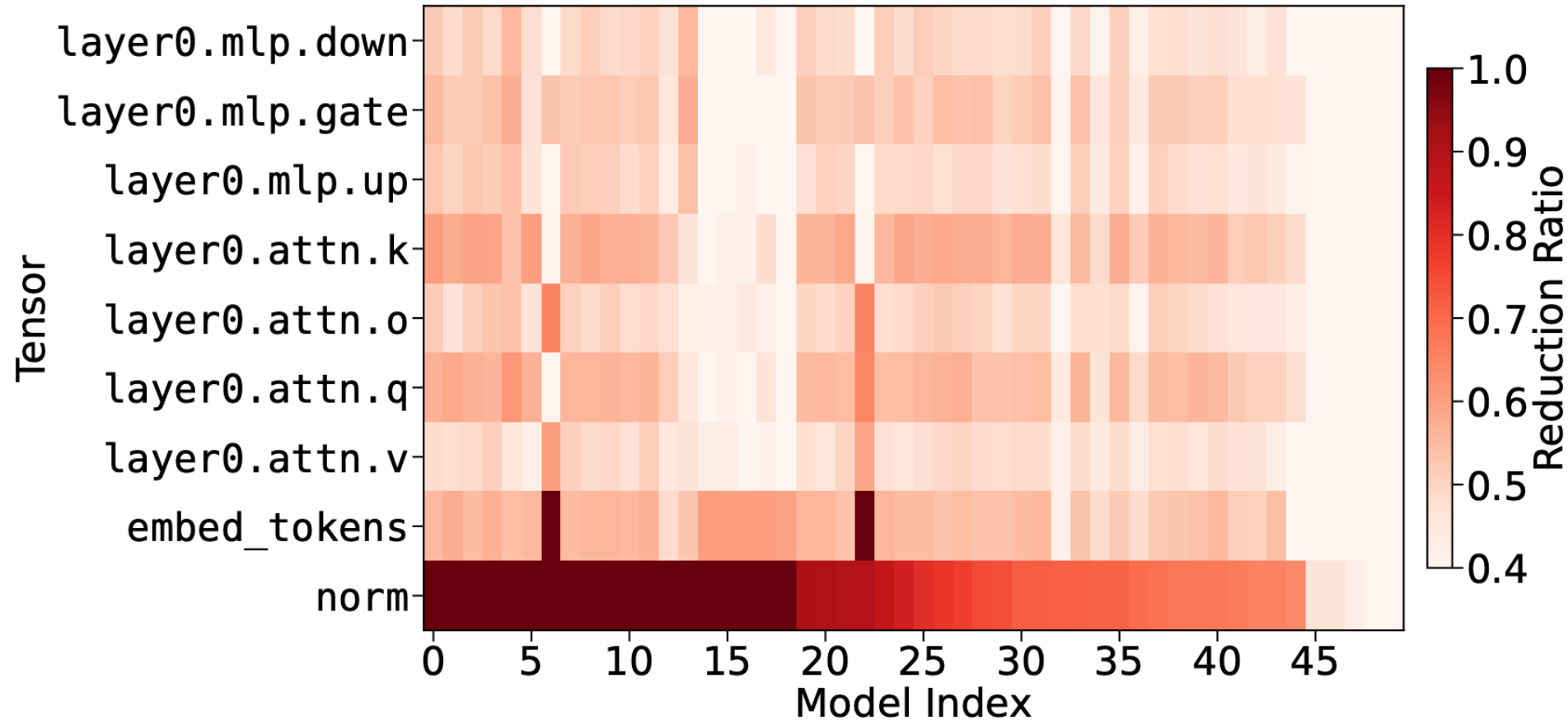
Real-world LLM lineage is complex



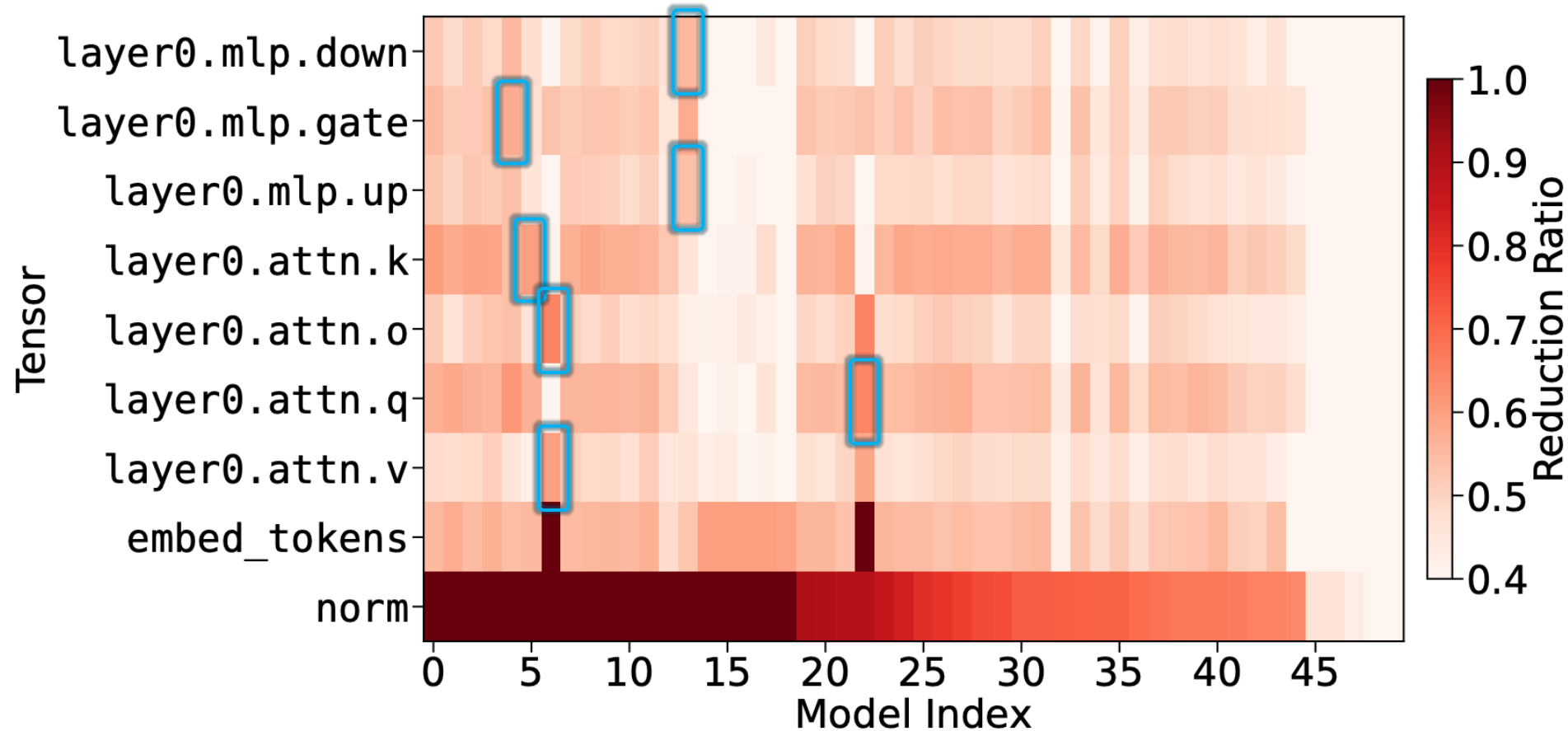
Overall reduction ratio: 47%

Finding 4: Real-world model lineage is multi-level, and compression is highly sensitive to pairing. Without accurate lineage, delta compression fails to eliminate all latent redundancy

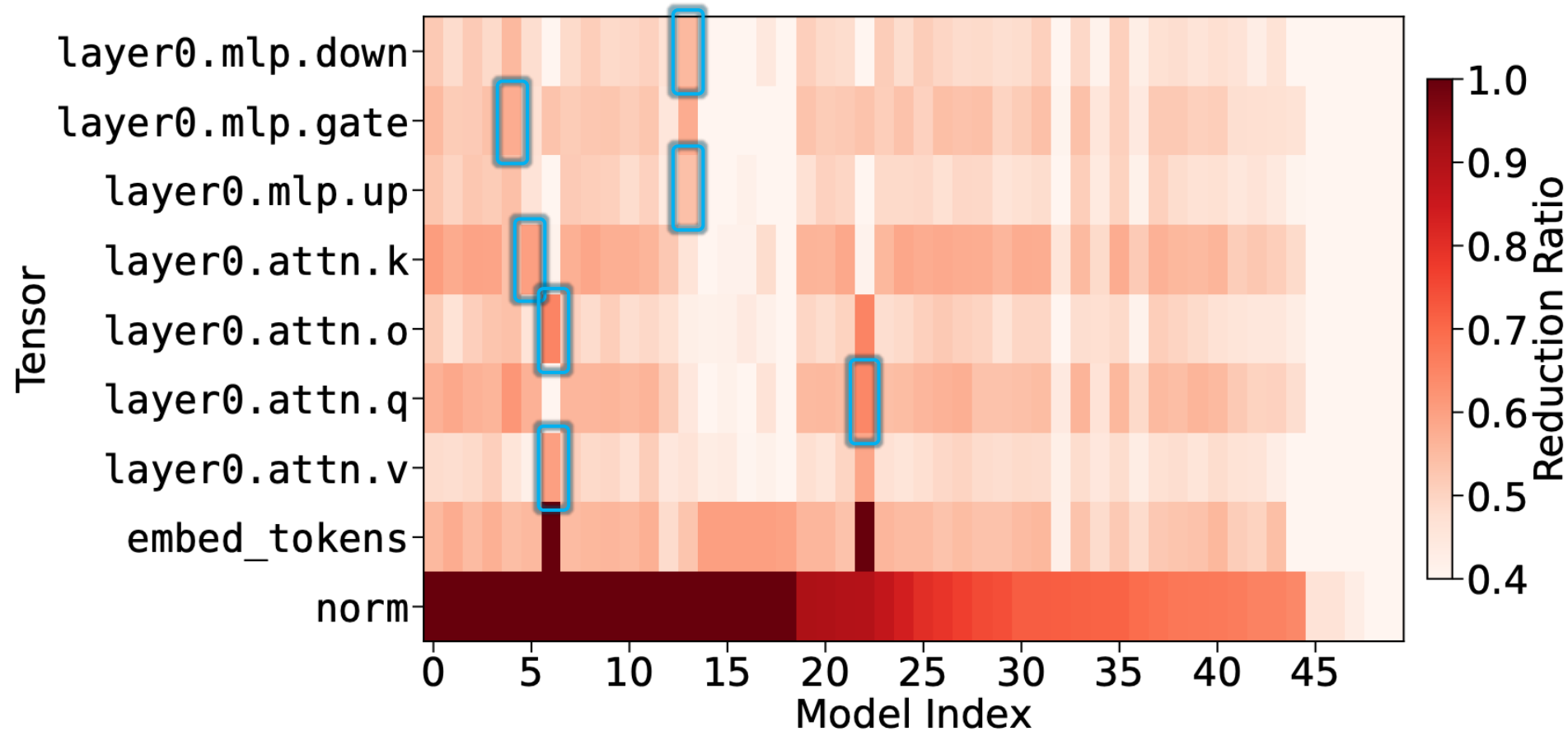
Tensor-level compressibility is heterogeneous



Tensor-level compressibility is heterogeneous

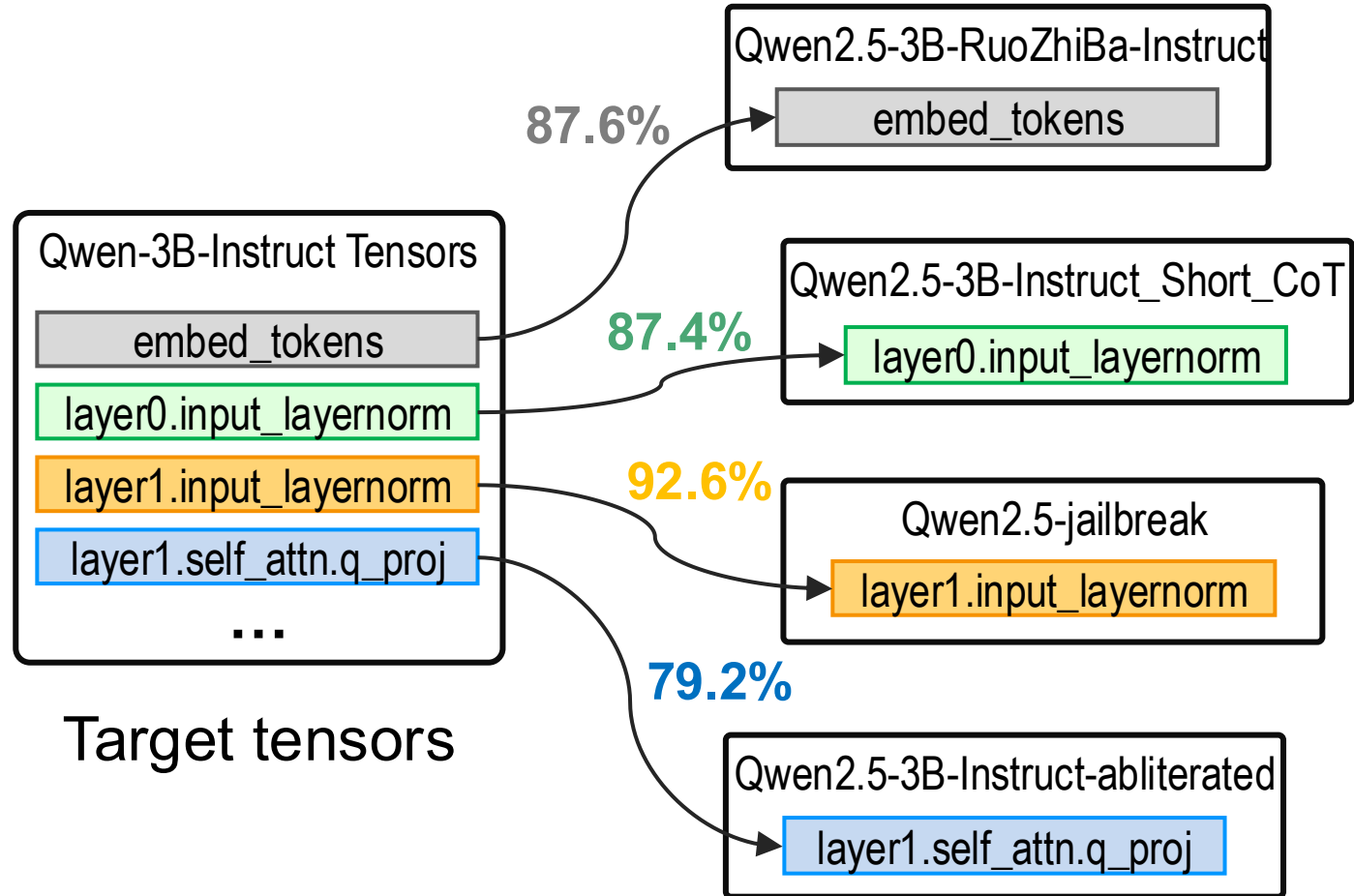


Tensor-level compressibility is heterogeneous



Finding 5: No single base model consistently serves as the best compression partner for all tensors of a target model

The best base tensor often lives in a different model



Target tensors

Best base tensors scattered over different models

Takeaway message: What we learned

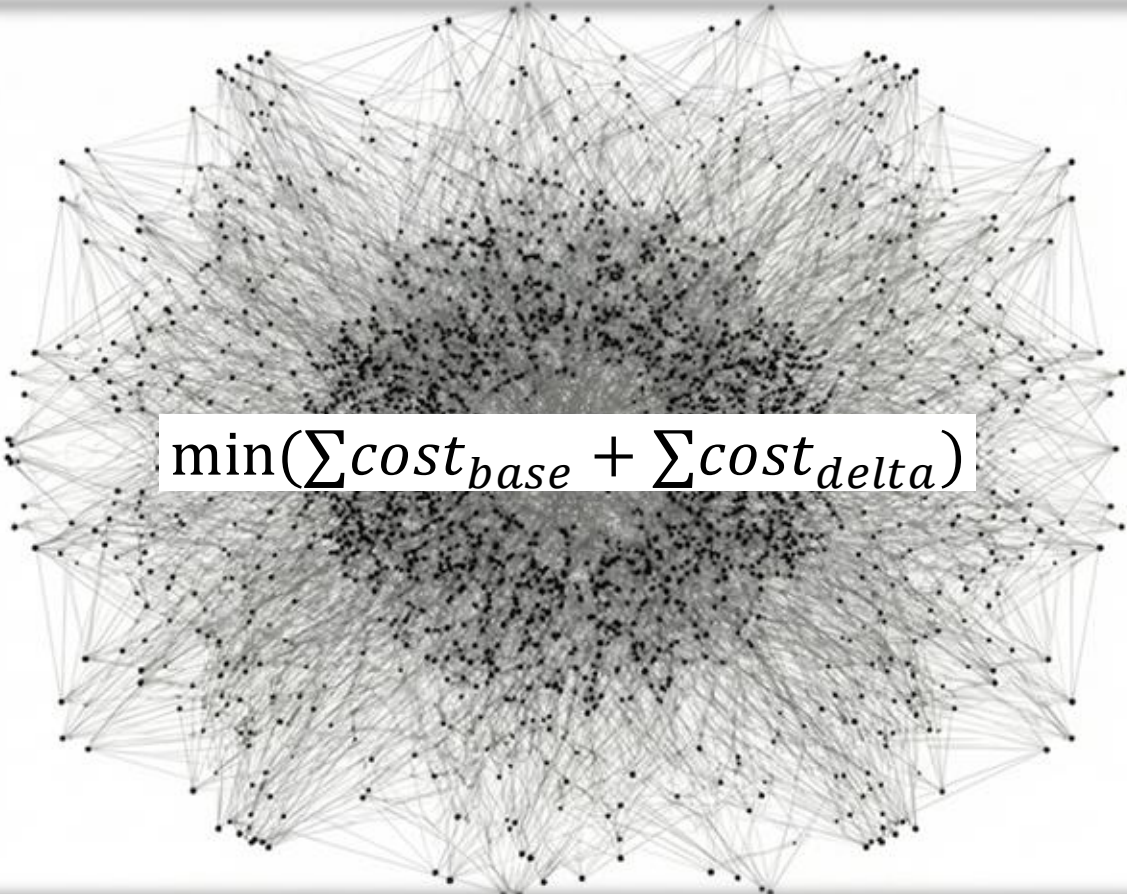
- **Finding 1:** **Fine-tuned** LLMs dominate storage cost
- **Finding 2:** LLM variants within a family are numerically similar and thus can benefit from **delta compression**
- **Finding 3:** User-provided metadata is unreliable, making **content-aware, model-generated metadata** a necessity
- **Finding 4:** Real-world model lineage is complex, and compression is **highly sensitive to pairing**
- **Finding 5:** **No single base model** consistently serves as the best compression partner for all tensors of a target model

Storage redundancy emerges at tensor level

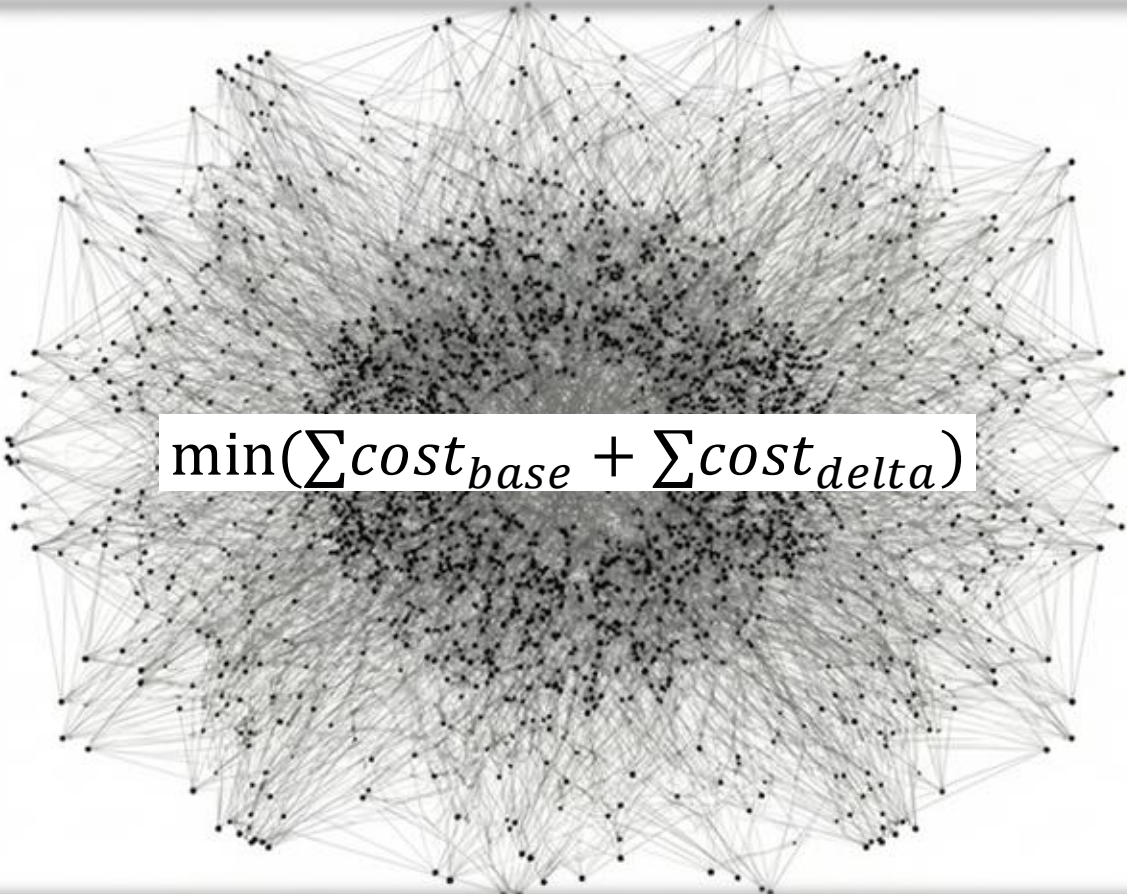
Finding optimal tensor pairs at scale is challenging

Problem complexity at scale

Billions of tensors across **millions** of large models lead to a **combinatorial explosion**, making exhaustive pairing computationally infeasible


$$\min(\sum cost_{base} + \sum cost_{delta})$$

Finding optimal tensor pairs at scale is challenging


$$\min(\sum cost_{base} + \sum cost_{delta})$$

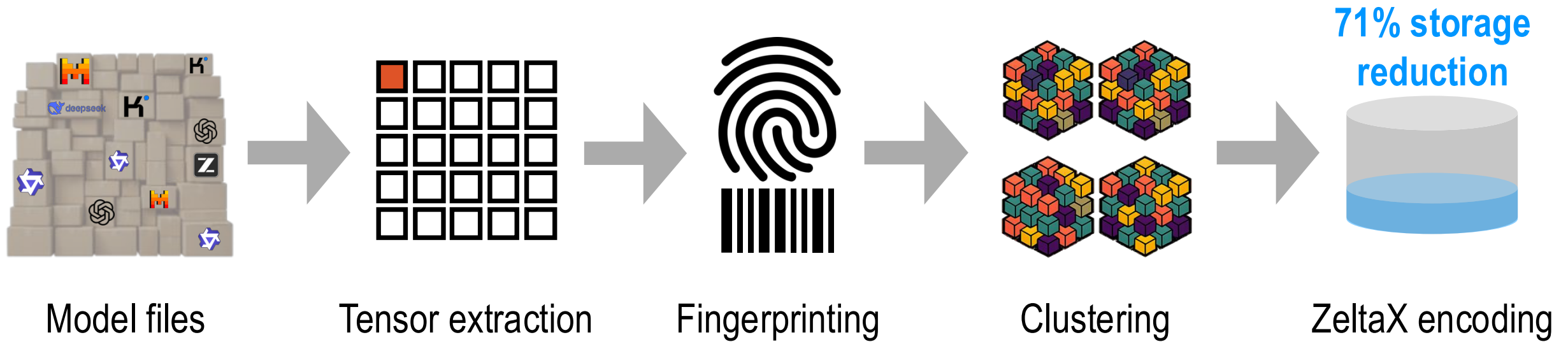
Problem complexity at scale

Billions of tensors across **millions** of large models lead to a **combinatorial explosion**, making exhaustive pairing computationally infeasible

I/O bottleneck at scale

Naïve inspection requires scanning **petabytes** of tensor data, incurring **expensive storage bandwidth cost**

TensorDex: A tensor-centric approach



1. TensorSketch

Robust, lightweight fingerprinting enables cheap compressibility prediction

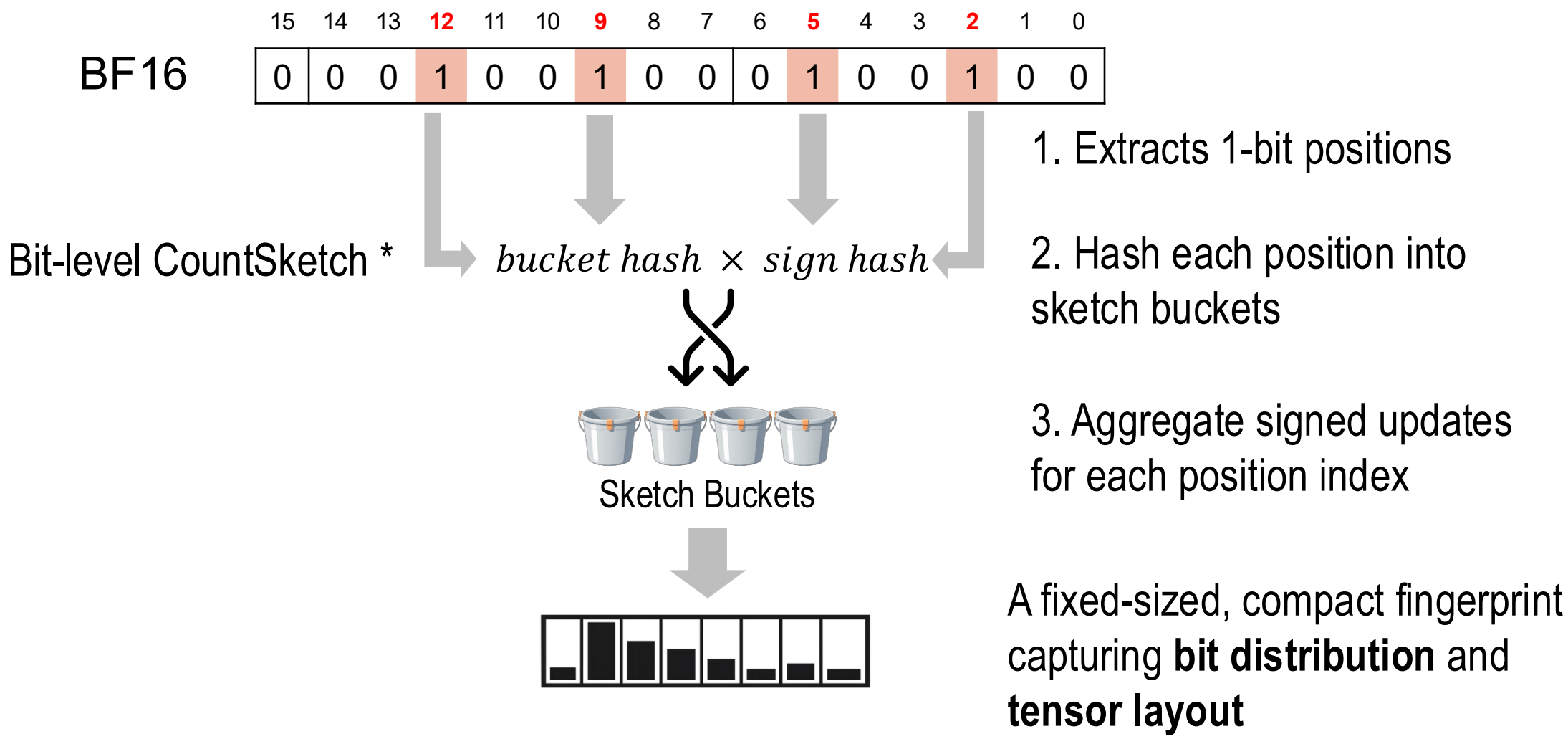
2. FlexSplit

Fast clustering without exact ILP solving for an NP-hard problem

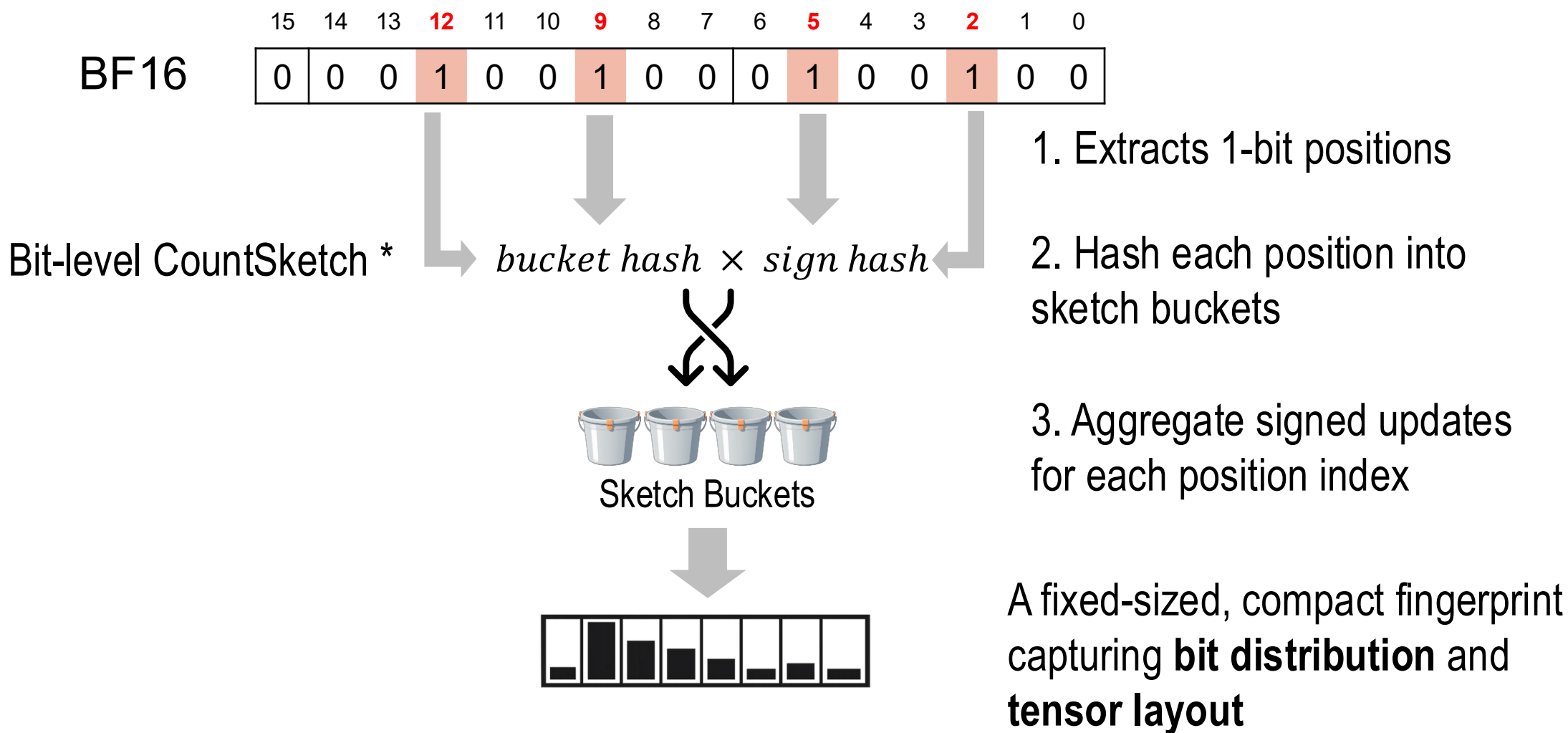
3. ZeltaX

Addressing BitX's numerical insensitivity & synergizing tensor clustering + delta compression

TensorSketch: Compression-aware tensor fingerprinting



TensorSketch: Compression-aware tensor fingerprinting



Enables instant compressibility estimation, avoiding excessive I/Os

FlexSplit: Fast & adaptive tensor clustering

Goal: Organizing petabytes of tensors into best compression clusters

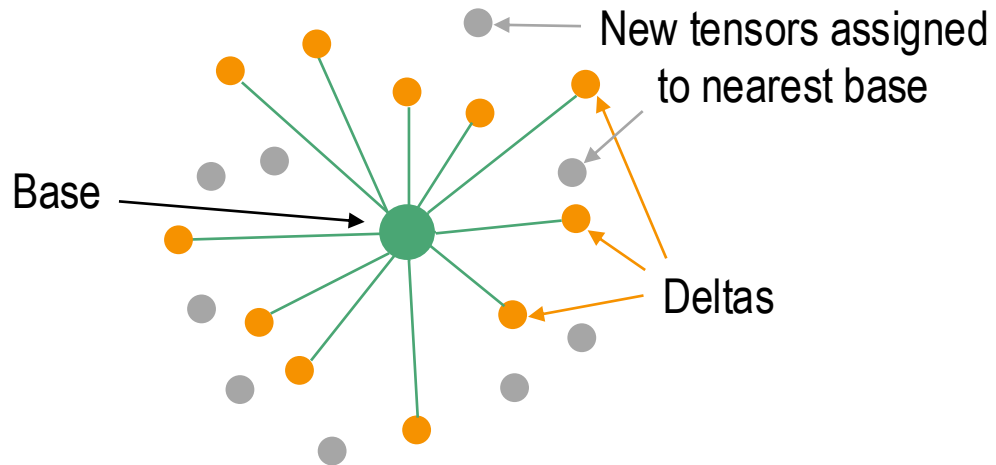
FlexSplit: Incrementally optimizes which tensors serve as “bases” and which are “deltas”

FlexSplit: Fast & adaptive tensor clustering

Goal: Organizing petabytes of tensors into best compression clusters

FlexSplit: Incrementally optimizes which tensors serve as “bases” and which are “deltas”

Phase 1: Greedy assignment



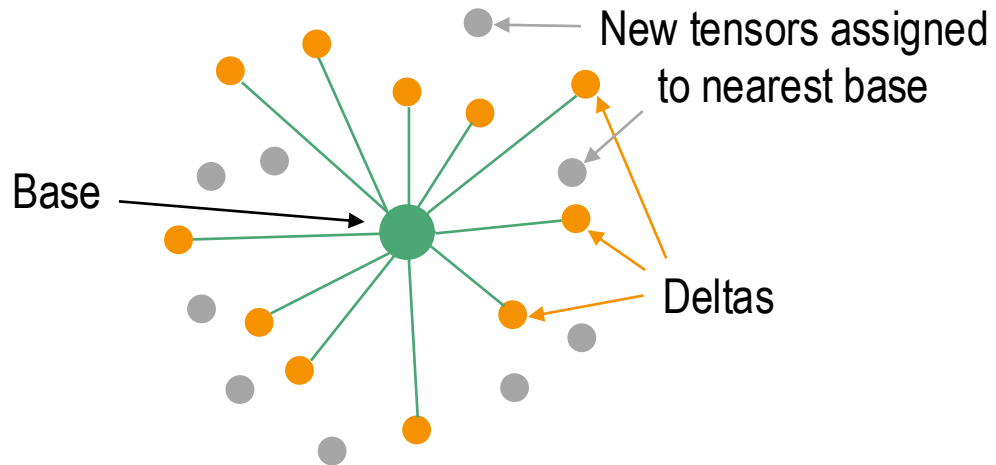
Tensors are assigned to nearest bases using TensorSketch, taking $O(k)$

FlexSplit: Fast & adaptive tensor clustering

Goal: Organizing petabytes of tensors into best compression clusters

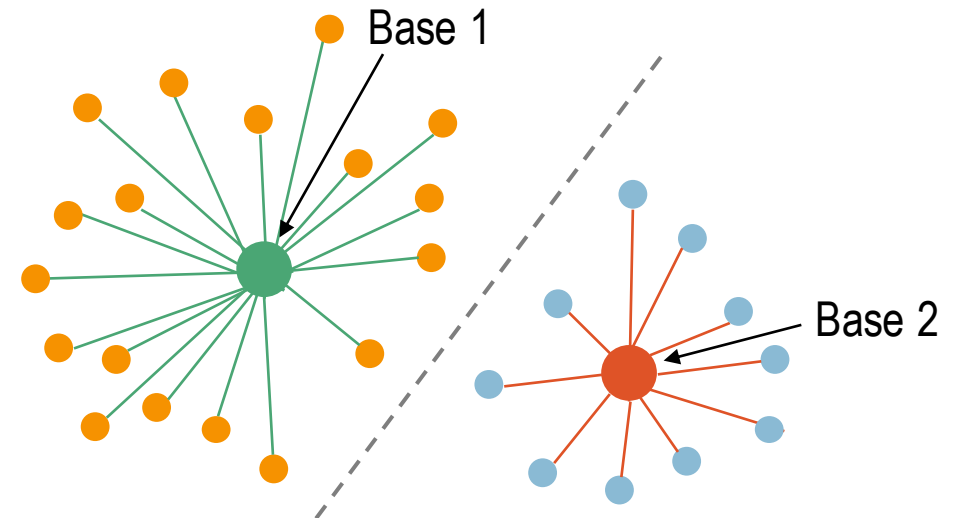
FlexSplit: Incrementally optimizes which tensors serve as “bases” and which are “deltas”

Phase 1: Greedy assignment



Tensors are assigned to nearest bases using TensorSketch, taking $O(k)$

Phase 2: Dynamic splitting



If cluster becomes too big with many tensors poorly compressed, split it and select a good candidate among all outliers to become a new base

From BitX to ZeltaX: BitX recap

Target (0.498046875)	00111110 11111111
Base (0.5)	00111111 00000000

BitX

Target	00111110 11111111
	$\oplus$ XOR
Base	00111111 00000000

↓

00000001 11111111

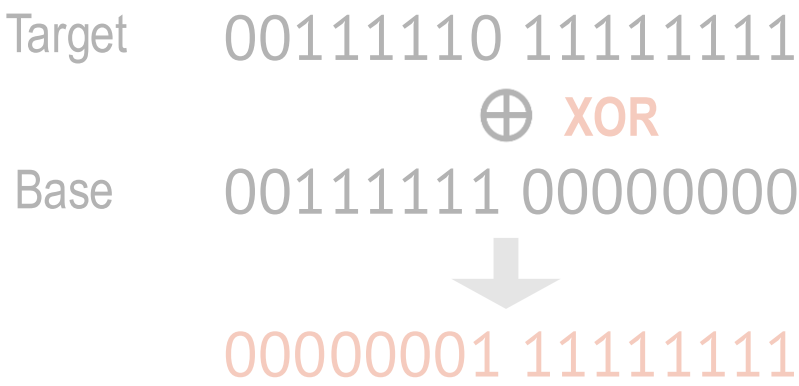
XOR sees bit flips, not numerical distance,
losing borrow & carry info and inflating entropy

Bit entropy = 7.91 bits/byte
High entropy / low compressibility

ZeltaX: Lowering delta entropy via numerical awareness

Target (0.498046875)	00111110	11111111
Base (0.5)	00111111	00000000

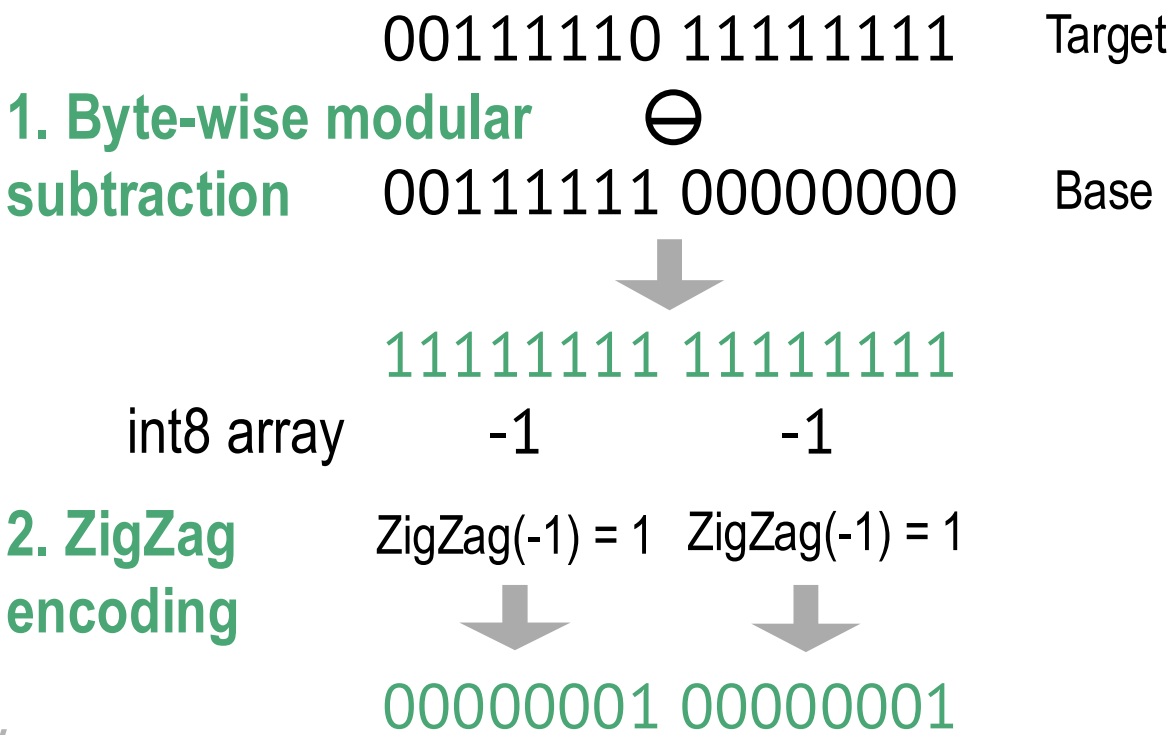
BitX



XOR sees bit flips, not numerical distance, losing borrow & carry info and inflating entropy

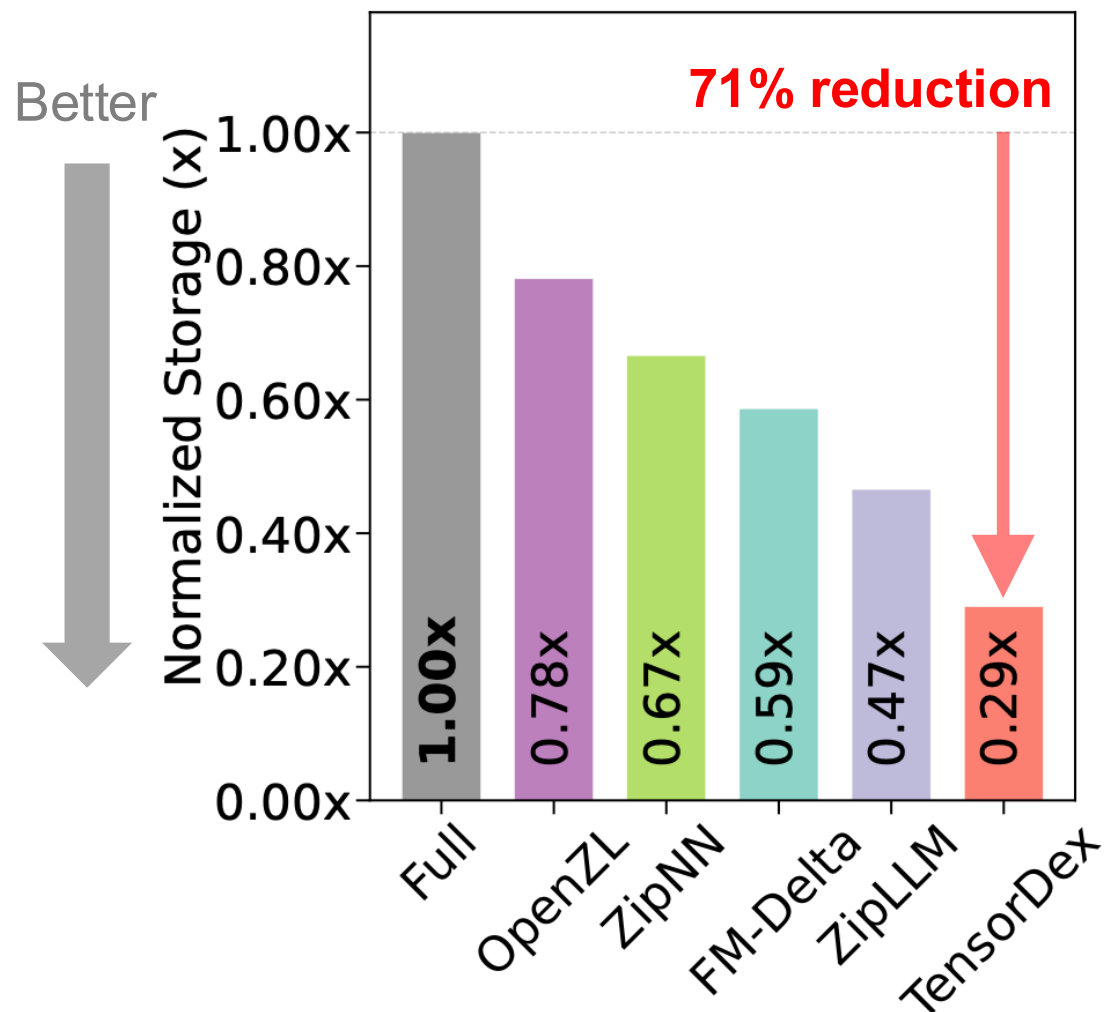
Bit entropy = 7.91 bits/byte
High entropy / low compressibility

ZeltaX



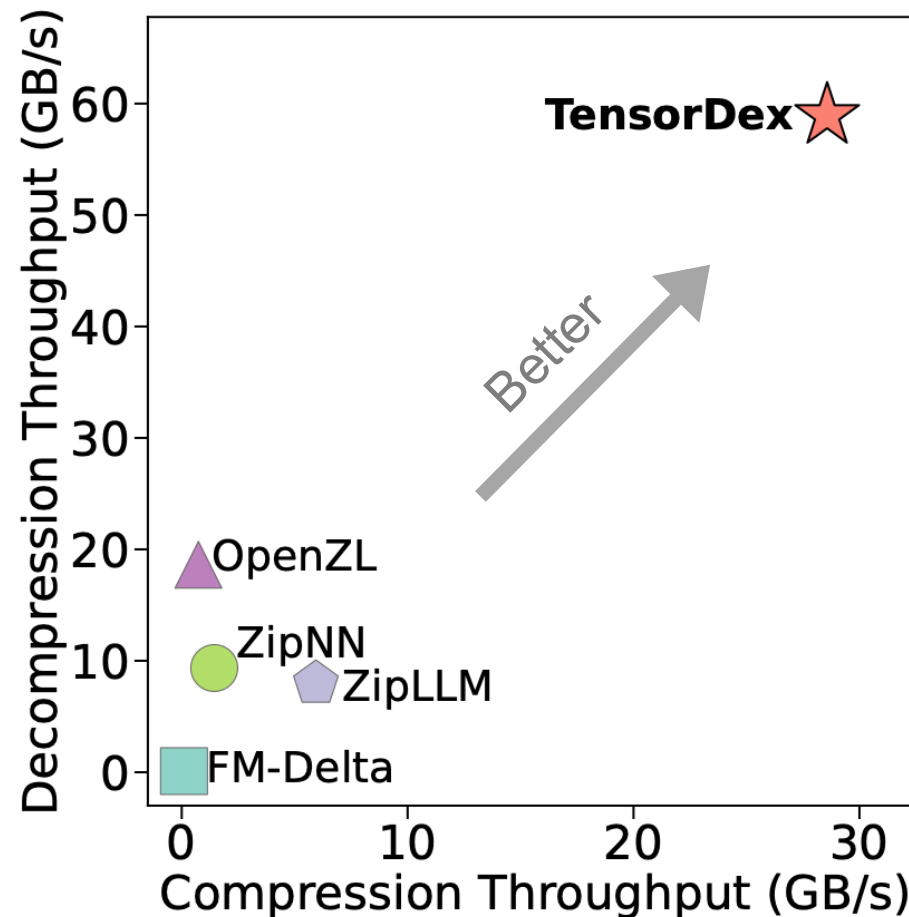
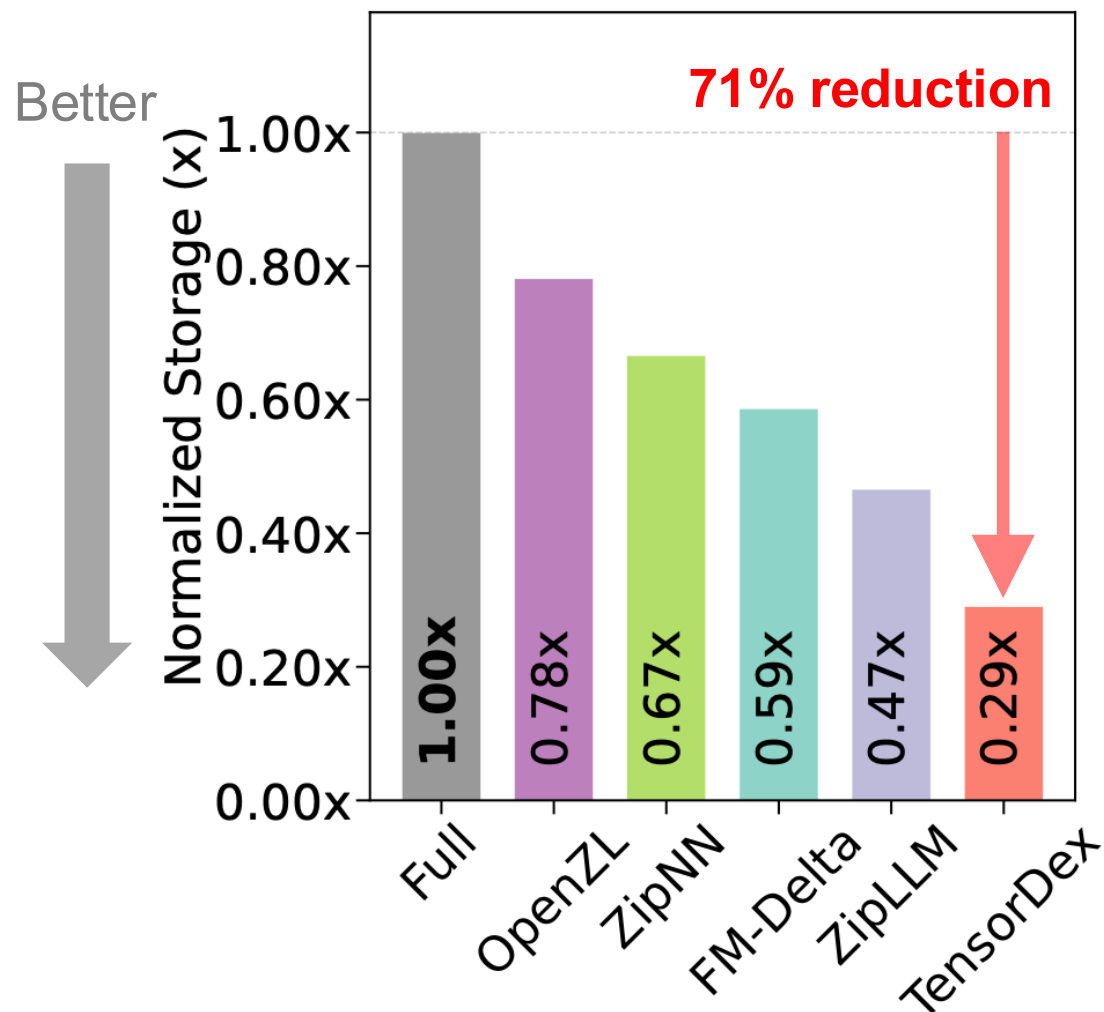
Bit entropy = 4.35 bits/byte
Low entropy / high compressibility

TensorDex evaluation



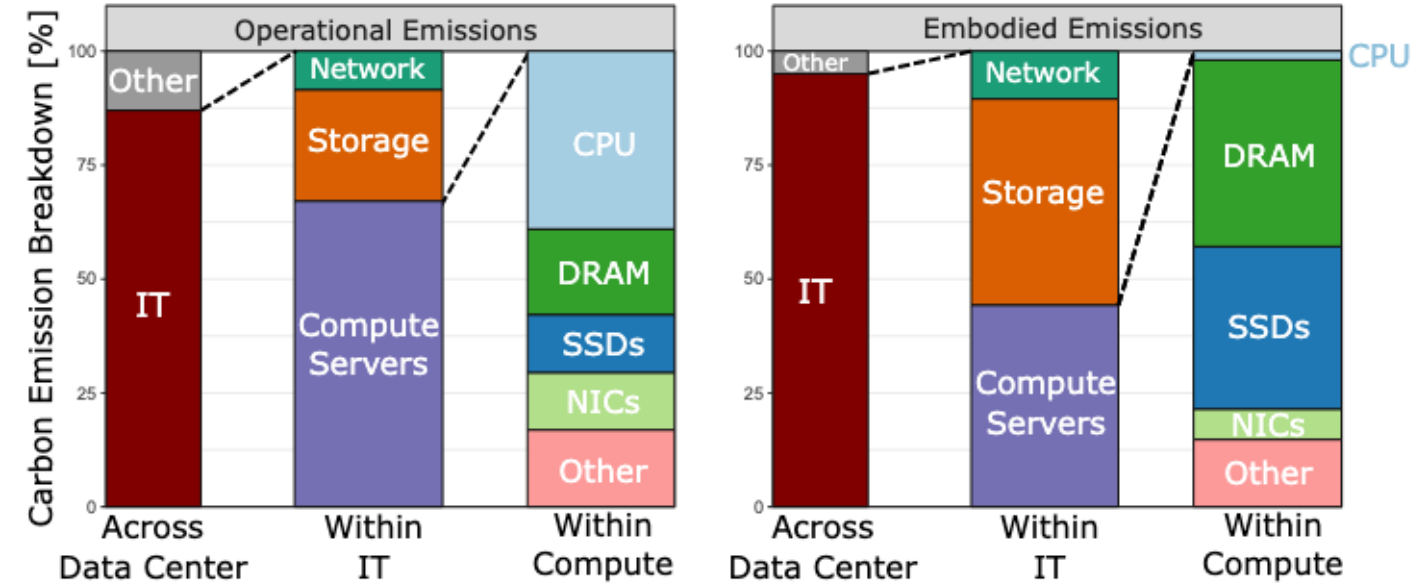
3,000+ randomly sampled Hugging Face LLM repos

TensorDex evaluation



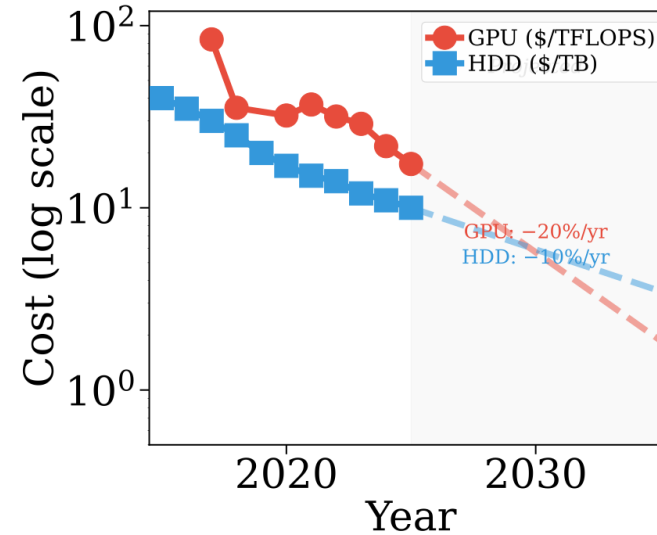
3,000+ randomly sampled Hugging Face LLM repos

Long-term storage sustainability problem

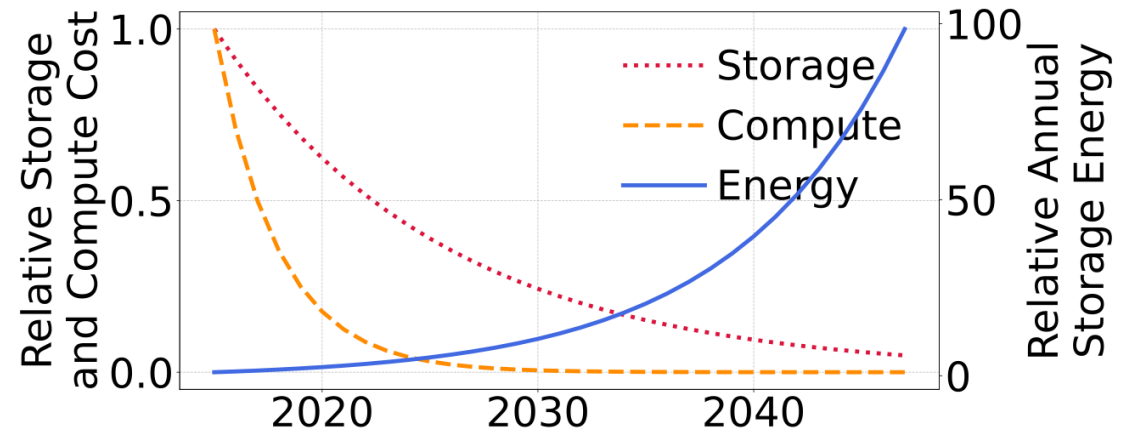


Carbon emission breakdown in general-purpose Azure datacenters *

While compute is transient, storage is persistent and exponentially accumulating, with ongoing energy/carbon cost.



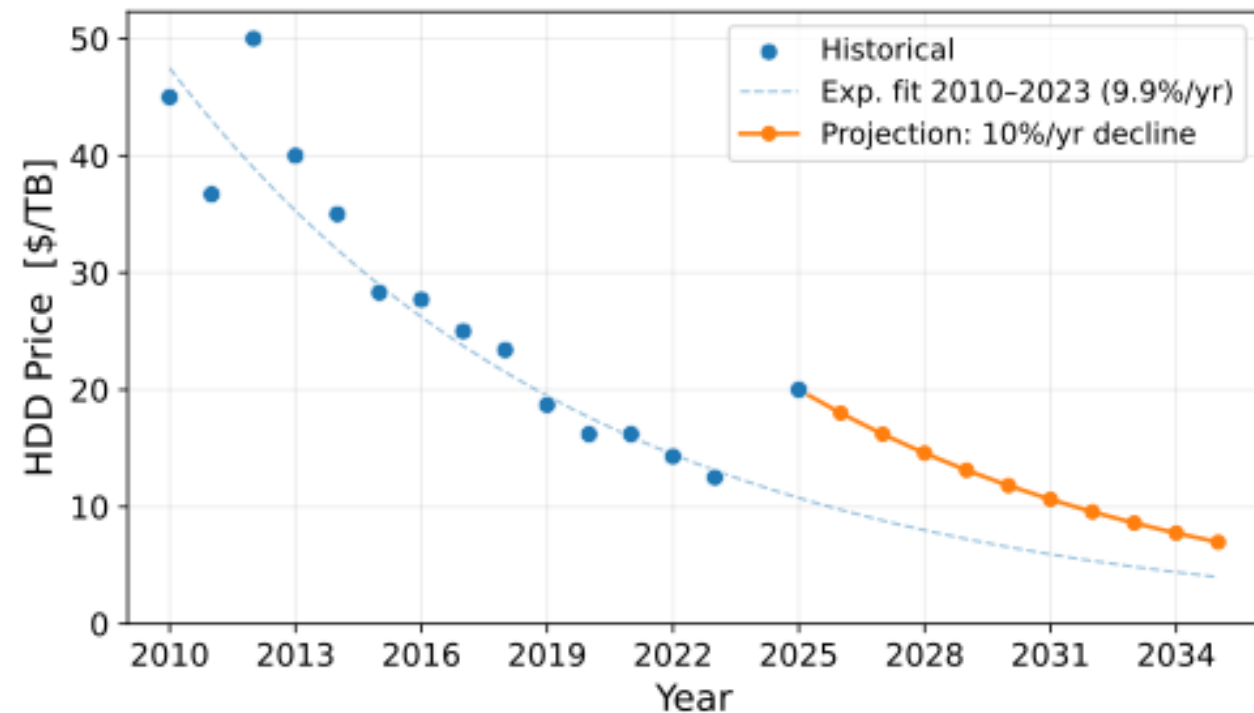
Historical and projected cost for GPU compute (\$/TFLOPS) and HDD storage (\$/TB)



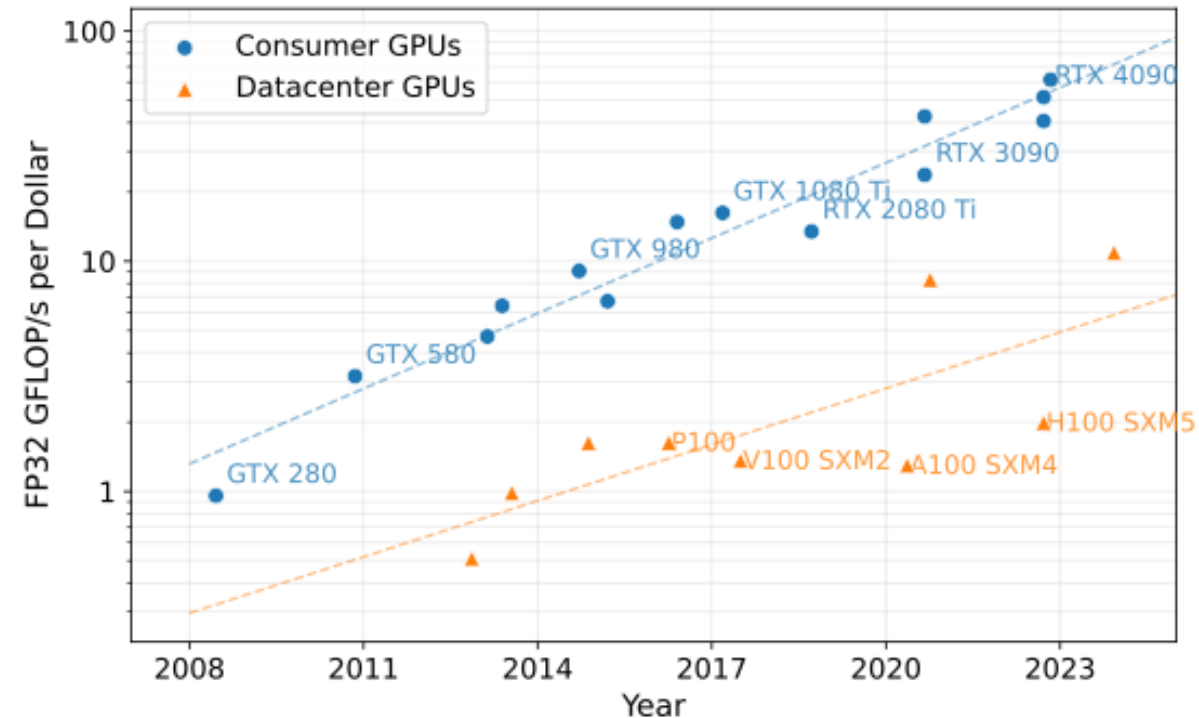
While compute and storage unit cost drops steadily over time, the accumulative nature of the exponentially growing storage consumption demands exponential increases in energy / carbon

*: Designing Cloud Servers for Lower Carbon. ISCA'24

Hardware cost trend



HDD \$ cost/TB
(10% yearly reduction)



GPU FP32 performance/\$
(30% yearly improvement)

Model storage cost

- Cost of 77PB of model storage
 - Physical storage capacity: $77\text{PB} \times 3 = 231\text{ PB}$
 - AWS S3: \$1.66M/month
 - Egress cost: tens of millions of dollars per month
 - Total energy usage: 109 MWh/month
 - Operational carbon: 0.51 ktCO₂e/year
 - Embodied carbon: 0.20 ktCO₂e/year amortized